

Tesi

Tommaso Moretta

Settembre 2025

Indice

1	Introduzione	4
1.1	Motivazioni e contesto	4
1.2	Obiettivi del progetto	4
1.3	Convenzioni Linguistiche	5
1.4	Contributi Progettuali e Ingegneristici	5
1.5	Struttura del documento	6
2	Contesto e Analisi del sistema	7
2.1	Il problema: frammentazione e inefficienza nella gestione dei concorsi	7
2.2	Stato dell'arte e Analisi delle Alternative	8
2.2.1	Strumenti Generici Adattati: lo "Status Quo"	8
2.2.2	Piattaforme per Domini Correlati (ma Differenti)	9
2.2.3	Posizionamento di Swift Contest	9
2.3	Attori e casi d'uso	10
2.3.1	Attori del Sistema	10
2.3.2	Casi d'Uso: Gestione Account e Funzionalità Generali	10
2.3.3	Casi d'Uso: Gestione del Ciclo di Vita del Contest	11
2.3.4	Casi d'Uso: Gestione delle Votazioni e dei Risultati	12
2.3.5	Casi d'Uso: Amministrazione del Sistema	13
2.4	Requisiti del sistema	13
2.4.1	Requisiti funzionali (Functionality)	14
2.4.2	Requisiti Non Funzionali	14
2.5	Modello del dominio	16
2.5.1	Entità Fondamentali	18
2.5.2	Vincoli Chiave del Dominio	18
3	Tecnologie e Strumenti Utilizzati	20
3.1	Stack Tecnologico Principale	20
3.1.1	Frontend: Flutter e Dart	20
3.1.2	Backend: Supabase, PL/pgSQL e TypeScript	21

3.2	Piattaforme, Strumenti e Servizi	21
3.2.1	Piattaforme di Deployment e Servizi Cloud	21
3.2.2	Strumenti di Sviluppo e Amministrazione	22
3.2.3	Version Control e Code Hosting	22
3.2.4	Strumenti di Documentazione e Modellazione	23
4	Progettazione e Implementazione del sistema	24
4.1	Linee guida progettuali	24
4.2	Architettura del frontend: il pattern MVVM con BLoC	25
4.2.1	Implementazione del ViewModel: il pattern BLoC	26
4.2.2	Navigazione con AutoRoute	28
4.2.3	Strategia di Navigazione e Passaggio dei Dati	30
4.2.4	Design System e Theming con Estensioni	32
4.3	Progettazione del Database	35
4.3.1	Schema Fisico e Tabelle Principali	35
4.3.2	Scelte di Design Chiave	36
4.3.3	Implicazioni del Design del Form di Voto sull'Aggregazione dei Risultati	39
4.4	Architettura del backend	40
4.4.1	Funzioni RPC (Remote Procedure Call)	40
4.4.2	Edge Functions	42
4.5	Architettura di Sicurezza: Principio del Minimo Privilegio e Accesso Mediato .	44
4.5.1	Il Principio del Minimo Privilegio e l'API come Gatekeeper	44
4.5.2	L'Eccezione Necessaria: le Policy RLS per il Servizio Realtime	45
4.5.3	Misure di Sicurezza Aggiuntive	45
4.6	Progettazione e Implementazione degli Strumenti di Amministrazione	47
4.6.1	Esigenza di un Pannello di Amministrazione	47
4.6.2	Scelta di Retool per lo Sviluppo Rapido	47
4.6.3	Architettura di Comunicazione Sicura tra Retool e Supabase	47
4.6.4	Funzionalità Implementate nel Pannello	51
5	Distribuzione e Prodotto finale	52
5.1	Strategia di distribuzione	52
5.2	Distribuzione Web su Vercel	53
5.3	Distribuzione Android	54
5.4	Backend e Supabase	54
5.5	Dashboard amministrativa (Retool)	54
5.6	Prodotto finale: Flussi Operativi e Interfaccia Utente	55
5.6.1	Flusso dell'Organizzatore	55
5.6.2	Flusso del Partecipante	56
5.6.3	Flusso del Giurato (Nominato e Semplice)	56

5.6.4	Flusso delle Notifiche	57
6	Conclusioni	58
6.1	Sintesi del lavoro svolto e risultati ottenuti	58
6.2	Limiti e analisi critica	59
6.3	Prospettive e sviluppi futuri	59
6.4	Considerazioni finali	60

Capitolo 1

Introduzione

1.1 Motivazioni e contesto

La gestione di **contest** in tempo reale rappresenta a oggi un processo frammentato e complesso. Organizzatori, partecipanti e giurati spesso ricorrono a strumenti diversi: servizi di posta elettronica per la comunicazione degli inviti, piattaforme di file sharing per la raccolta dei lavori, fogli di calcolo per la tabulazione dei punteggi.

Questa frammentazione genera inefficienze operative, rischi di errore umano e una significativa difficoltà nel garantire trasparenza e sicurezza. L'analisi dello stato dell'arte, come verrà dettagliato nel capitolo successivo, ha inoltre rivelato un **significativo vuoto di mercato**: non esistono soluzioni verticali e accessibili che coprano in modo integrato l'intero ciclo di vita di un contest.

Il progetto **Swift Contest** nasce proprio per colmare questa lacuna, proponendo un sistema moderno, sicuro e multipiattaforma che unisce in un'unica applicazione le principali fasi del ciclo di vita di un contest: dalla creazione e configurazione, all'invito dei partecipanti e dei giurati, fino alla raccolta dei lavori e alla gestione delle sessioni di voto in tempo reale.

1.2 Obiettivi del progetto

L'obiettivo primario di questo lavoro è stato quello di **progettare e implementare l'architettura** di un sistema completo di *contest management*, adottando tecnologie moderne e approcci architetturali rigorosi.

Gli obiettivi specifici, che hanno guidato la progettazione, possono essere così sintetizzati:

- **Sviluppo Cross-Platform**: garantire un'esperienza utente fluida e unificata su più piattaforme (mobile e web) attraverso lo sviluppo di un client basato su un'unica codebase.

- **Sicurezza "by Design"**: adottare un'architettura di sicurezza basata sul **Principio del Minimo Privilegio (Principle of Least Privilege)**, dove l'accesso al database è negato per default e concesso selettivamente solo tramite un'API controllata.
- **Esperienza Utente Mirata**: progettare interfacce utente intuitive e funzionali, modellate sui ruoli specifici dei diversi attori del sistema (organizzatore, partecipante, giurato).
- **Accesso Flessibile alla Votazione**: implementare un sistema di voto che supporti due modalità di accesso per i giurati. Oltre ai giurati nominati (utenti registrati e invitati in giurie di tipo *nominato (appointed)*), il sistema deve permettere la partecipazione di **giurati semplici**. Questo avviene tramite la creazione, da parte dell'organizzatore, di una **giuria di tipo semplice (simple)** a cui è associato un token di accesso sicuro, abbassando così la barriera d'ingresso per i giurati occasionali (siano essi ospiti o utenti già registrati).
- **Affidabilità del Processo**: assicurare l'integrità e la verificabilità del processo di voto attraverso meccanismi tecnici robusti, come lo *snapshot* dei dati al momento dell'avvio della sessione e la possibilità di applicare restrizioni basate sulla geo-localizzazione.

1.3 Convenzioni Linguistiche

Al fine di garantire la massima chiarezza e coerenza con gli standard del settore, nel presente elaborato è stata adottata una convenzione linguistica specifica per gli elementi prettamente tecnici.

Per tutti gli artefatti di progettazione e implementazione, come i **diagrammi UML**, le **figure architetturali**, i **listati di codice** e i nomi delle componenti software, è stato scelto di utilizzare esclusivamente la **lingua inglese**. Questa decisione riflette lo standard de facto della comunità informatica internazionale, assicurando che la documentazione tecnica sia immediatamente comprensibile e allineata alle convenzioni del settore. Nel testo descrittivo, termini italiani e inglesi potranno essere usati come sinonimi dove appropriato (es. Contest/Concorso).

1.4 Contributi Progettuali e Ingegneristici

Il valore principale di questo lavoro di tesi risiede nell'applicazione rigorosa di principi di ingegneria del software moderni per risolvere un problema pratico. Il contributo non è di natura teorica, ma **progettuale e architetturale**.

Il progetto **Swift Contest** si presenta come un *blueprint*, ovvero un modello di riferimento, per la creazione di applicazioni collaborative complesse e sicure. I contributi specifici sono:

- **Proposta di una Nuova Categoria di Prodotto:** data l'assenza di soluzioni verticali, il progetto stesso si configura come un caso di studio per una nuova categoria di software dedicato alla gestione dei contest, colmando un vuoto di mercato.
- **Dimostrazione di un'Architettura Sicura su BaaS:** Il sistema dimostra come implementare un'architettura basata sul Principio del Minimo Privilegio utilizzando una piattaforma Backend-as-a-Service (Supabase).
- **Modello Dati Robusto per Processi Immutabili:** Viene presentato un design del database che garantisce l'integrità e l'attendibilità dei processi di voto tramite l'uso di "attributi di snapshot" e denormalizzazione controllata.

In sintesi, il valore della tesi è nel dimostrare come, partendo da un'esigenza reale, sia stato possibile progettare e realizzare una soluzione completa, sicura e ben ingegnerizzata, utilizzando strumenti accessibili e seguendo le best practice del settore.

1.5 Struttura del documento

Il presente elaborato è organizzato in cinque capitoli principali:

- **Capitolo 1 – Introduzione:** illustra le motivazioni, gli obiettivi di alto livello e il valore del lavoro di tesi.
- **Capitolo 2 – Contesto e Analisi del sistema:** definisce in dettaglio il problema, analizza lo stato dell'arte e presenta i requisiti del sistema.
- **Capitolo 3 – Progettazione e Implementazione:** descrive l'architettura software, il design del database e l'implementazione delle principali funzionalità.
- **Capitolo 4 – Distribuzione e Prodotto finale:** presenta il processo di deploy, l'interfaccia utente e i flussi di utilizzo reali.
- **Capitolo 5 – Conclusioni:** riflette sui risultati ottenuti, ne analizza i limiti e propone possibili sviluppi futuri.

Capitolo 2

Contesto e Analisi del sistema

2.1 Il problema: frammentazione e inefficienza nella gestione dei concorsi

La gestione di un contest, specialmente in ambiti creativi, è un processo che coinvolge una molteplicità di attori (organizzatori, partecipanti, giurati) e fasi operative sequenziali e interdipendenti. L'approccio tradizionale si affida a un assemblaggio di strumenti generici e non integrati, creando un flusso di lavoro discontinuo, inefficiente e prone all'errore.

Le criticità di questo modello possono essere analizzate nel dettaglio:

- **Frammentazione del processo e dei dati:** la comunicazione degli inviti avviene tramite servizi di posta elettronica, la raccolta dei lavori tramite piattaforme di cloud storage (es. Google Drive, Dropbox) e la tabulazione dei voti tramite fogli di calcolo (es. Microsoft Excel). Questo approccio disperde le informazioni, rende difficile avere una visione d'insieme e richiede un costante lavoro manuale di sincronizzazione e consolidamento dei dati.
- **Mancanza di integrità e consistenza dei dati:** la gestione manuale aumenta esponenzialmente il rischio di errori. Esempi comuni includono la perdita di iscrizioni, l'errata trascrizione dei punteggi o l'inclusione di partecipanti non idonei. Non esiste un'unica "fonte di verità" per i dati del contest.
- **Carenza di strumenti in tempo reale:** l'assenza di un sistema centralizzato impedisce di gestire sessioni di voto live in modo coordinato e trasparente.
- **Rischi di sicurezza e potenziale di abuso:** la sottomissione di voti multipli, la partecipazione di utenti non invitati e la gestione di conflitti di interesse diventano problemi concreti e di difficile gestione.

L'obiettivo primario del sistema **Swift Contest** è superare queste criticità strutturali, fornendo un ambiente end-to-end integrato, sicuro e automatizzato, che funga da unica fonte di verità per l'intero ciclo di vita del concorso.

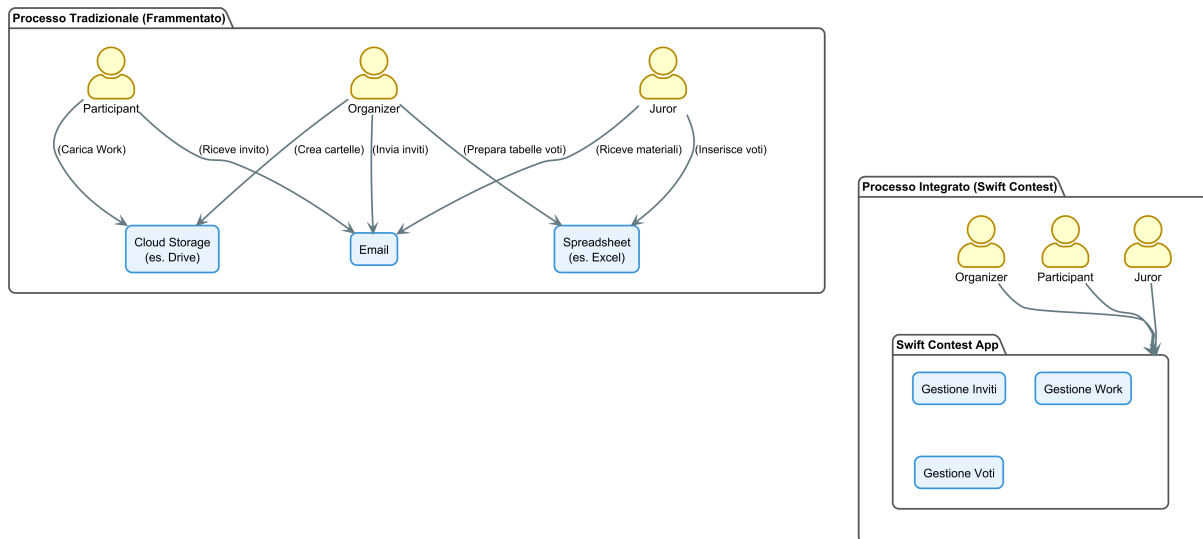


Figura 2.1: Confronto tra il flusso di lavoro tradizionale e l'approccio integrato di Swift Contest.

2.2 Stato dell'arte e Analisi delle Alternative

L'analisi dello stato dell'arte ha rivelato un'interessante lacuna nel panorama software attuale: **non esistono soluzioni verticali e accessibili che coprano in modo integrato l'intero ciclo di vita di un contest**. Le pratiche correnti si affidano a due approcci principali, entrambi inadeguati.

2.2.1 Strumenti Generici Adattati: lo "Status Quo"

Questa è la pratica più diffusa e rappresenta il problema fondamentale che **Swift Contest** si propone di risolvere. Consiste nell'utilizzare una combinazione di strumenti generici, non pensati per questo scopo, creando un flusso di lavoro frammentato e di difficile gestione, ad esempio:

- **Google Forms** per la raccolta delle iscrizioni.
- **Google Drive** per la sottomissione dei lavori.
- **Google Sheets** per la tabulazione manuale dei voti.
- **Email** per tutte le comunicazioni.

Come già analizzato, questo approccio soffre di gravi criticità in termini di sicurezza, integrità dei dati ed efficienza operativa, che hanno motivato la nascita del progetto.

2.2.2 Piattaforme per Domini Correlati (ma Differenti)

Sono state analizzate anche piattaforme software specializzate in domini vicini a quello dei contest, per verificare la possibilità di un loro adattamento. Tuttavia, nessuna di esse si è rivelata una soluzione praticabile.

Piattaforme di e-voting

Sistemi come *Helios Voting* o soluzioni basate su blockchain sono progettati per il **voto politico o assembleare**. Il loro focus è sulla crittografia e sull'anonimato del votante. Di conseguenza, mancano completamente delle funzionalità necessarie per un contest, quali:

- La possibilità di invitare partecipanti e creare giurie.
- La sottomissione e gestione di opere digitali.
- La creazione di moduli di voto personalizzati con campi testuali e numerici a punteggio.

Il loro scopo è intrinsecamente diverso e non sono in alcun modo utilizzabili.

Piattaforme per la gestione di eventi

Soluzioni commerciali come *Eventbrite* sono focalizzate sulla **gestione logistica di eventi e sulla vendita di biglietti**. Sebbene eccellano nella gestione delle registrazioni, non offrono alcuna funzionalità specifica per il dominio dei contest, come:

- La sottomissione di opere digitali.
- La creazione di giurie con ruoli definiti.
- La configurazione di moduli di valutazione strutturati.
- La gestione di sessioni di voto in tempo reale.

Il loro scopo è completamente diverso e non sono in alcun modo adattabili.

2.2.3 Posizionamento di Swift Contest

L'analisi conferma che Swift Contest si posiziona in un **vuoto di mercato e tecnologico**. Non mira a competere con le piattaforme di e-voting o di gestione eventi, ma a creare una **nuova categoria di prodotto**: una soluzione verticale, integrata e accessibile, progettata specificamente per le esigenze di organizzatori, partecipanti e giurati di un contest.

2.3 Attori e casi d'uso

Il sistema è progettato per servire diversi tipi di utenti, ciascuno con un ruolo e un insieme di permessi ben definiti. L'interazione tra questi attori e le funzionalità del sistema è stata modellata attraverso una serie di diagrammi dei Casi d'Uso, suddivisi in pacchetti logici (packages) che legano funzionalità della stessa categoria per migliorarne leggibilità e modularità.

2.3.1 Attori del Sistema

L'analisi del dominio ha permesso di identificare i seguenti attori principali:

- **Guest:** rappresenta un utente non autenticato. In questo sistema, che non prevede contenuti pubblici, un *Guest* non ha privilegi di visualizzazione. Le sue uniche capacità sono quelle di **autenticarsi** (tramite registrazione o login) o di agire come **Simple Juror** se in possesso di un token di accesso.
- **Authenticated:** è un attore astratto che rappresenta un utente che ha completato l'autenticazione. Funge da base per i ruoli operativi, che ne ereditano le funzionalità comuni come la gestione del proprio profilo e delle notifiche.
- **Organizer:** è un utente autenticato responsabile della creazione e della gestione completa di un contest.
- **Participant:** è un utente autenticato che si iscrive a un contest per sottoporre il proprio lavoro.
- **Juror:** è un utente autenticato il cui ruolo primario è quello di far parte di una giuria nominata (*appointed jury*), oppure votare come *Simple Juror* se in possesso di un token per la votazione.
- **Admin:** è un ruolo di amministrazione, gestito tramite una dashboard esterna (Retool), con privilegi elevati per la manutenzione del sistema.

2.3.2 Casi d'Uso: Gestione Account e Funzionalità Generali

Questa area copre le funzionalità di base legate all'accesso e alla gestione dell'account utente. Include i processi di autenticazione, la modifica del profilo personale e la gestione delle notifiche interne.

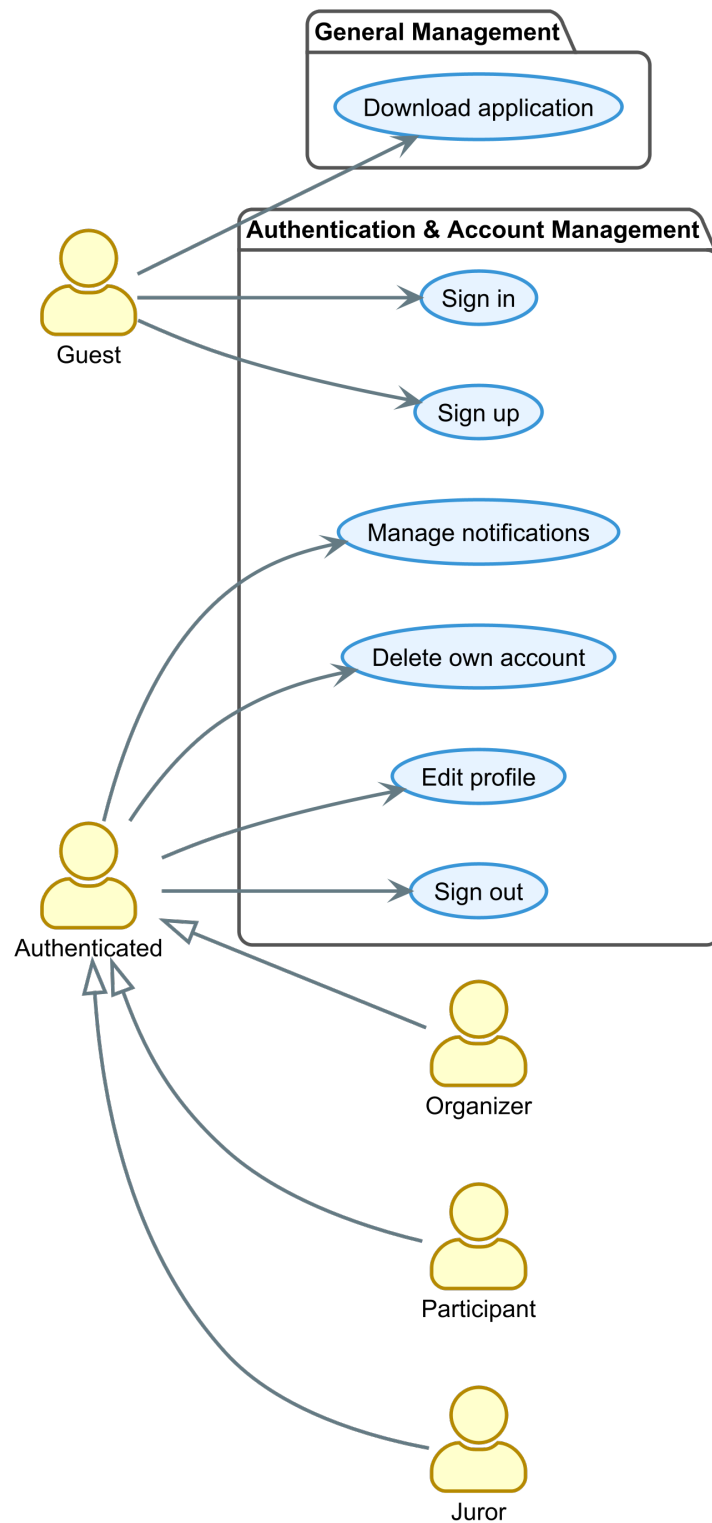


Figura 2.2: Diagramma dei Casi d'Uso: Gestione Account e Funzionalità Generali.

2.3.3 Casi d'Uso: Gestione del Ciclo di Vita del Contest

Questa è l'area funzionale più vasta e descrive il cuore del processo organizzativo. Copre la creazione di un contest, la gestione dei suoi membri (partecipanti e giurati) e la sottomissione

dei lavori. Gli attori principali in questo contesto sono l' *Organizer* e il *Participant*.

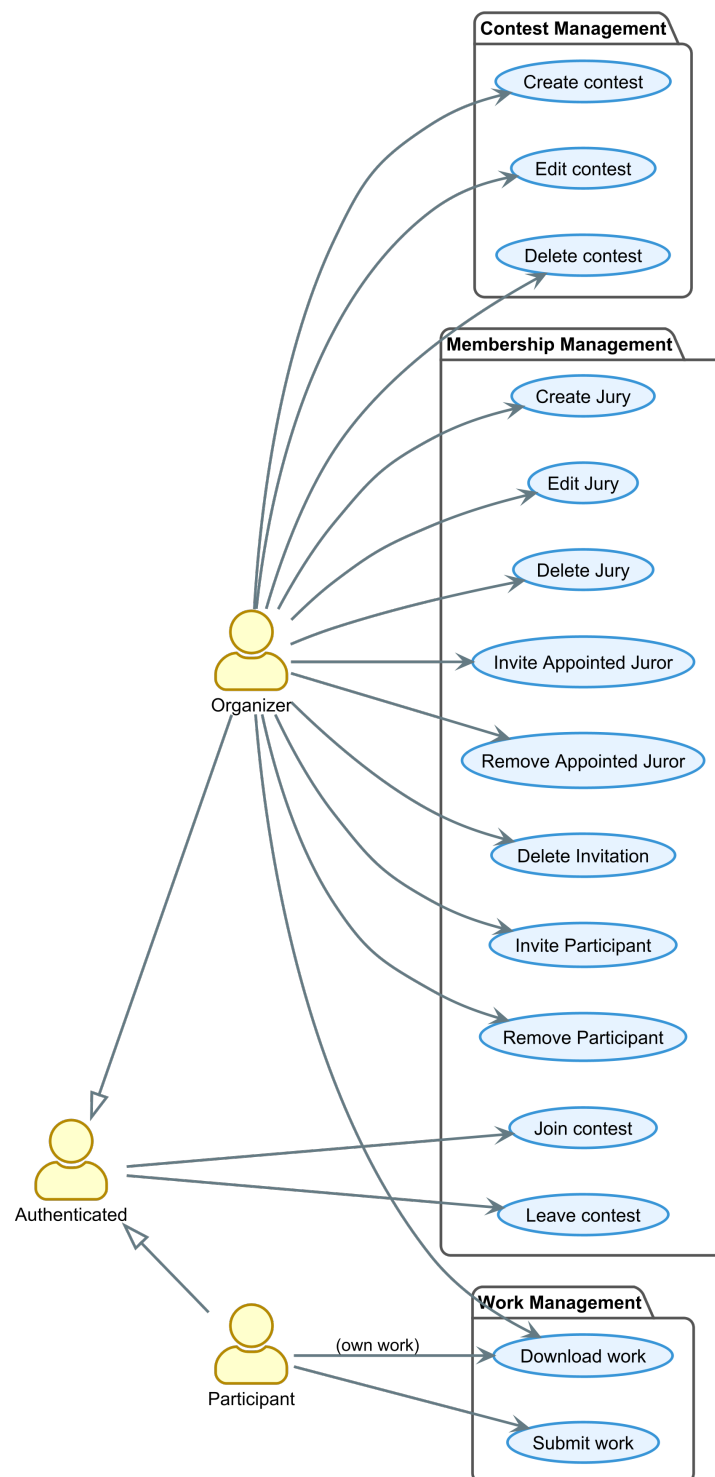


Figura 2.3: Diagramma dei Casi d'Uso: Gestione del Ciclo di Vita del Contest.

2.3.4 Casi d'Uso: Gestione delle Votazioni e dei Risultati

Questa sezione si concentra interamente sul processo critico della votazione. Include la configurazione del form di voto, la conduzione della sessione di voto (anche con restrizioni geografiche)

e la generazione e pubblicazione delle classifiche. Gli attori principali qui sono l' *Organizer* e il *Juror*.

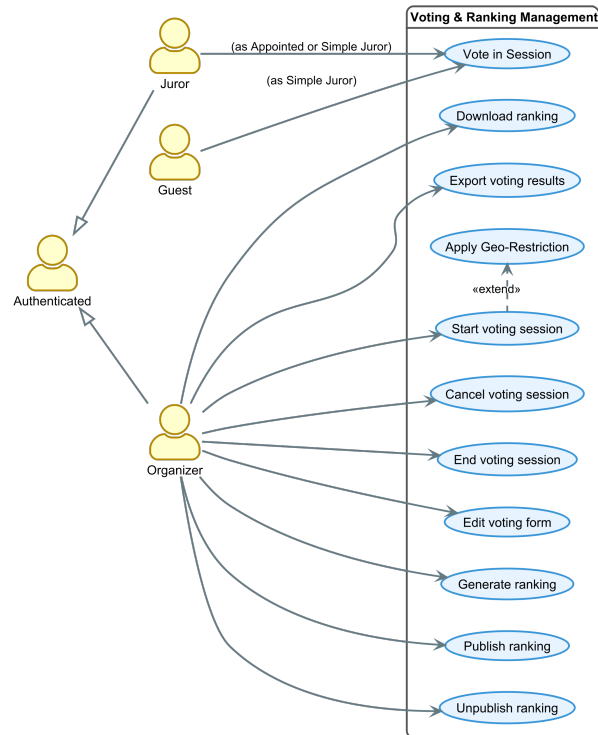


Figura 2.4: Diagramma dei Casi d'Uso: Gestione delle Votazioni e dei Risultati.

2.3.5 Casi d'Uso: Amministrazione del Sistema

Quest'ultima area isola le funzionalità di amministrazione, che non sono accessibili tramite l'applicazione principale ma attraverso una dashboard esterna. Queste operazioni sono riservate all'attore *Admin* e sono fondamentali per la manutenzione e la moderazione della piattaforma.

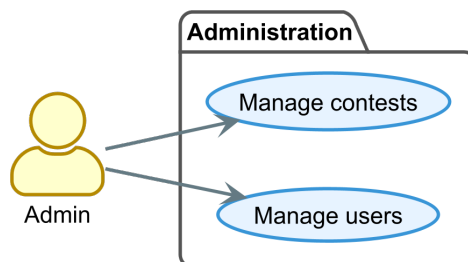


Figura 2.5: Diagramma dei Casi d'Uso: Amministrazione del Sistema.

2.4 Requisiti del sistema

I requisiti del sistema sono stati classificati secondo il modello **FURPS+**, un'estensione del modello FURPS che include categorie aggiuntive per coprire in modo esaustivo tutti i vincoli

di un progetto software.

2.4.1 Requisiti funzionali (Functionality)

- **RF-01: Gestione Contest:** Il sistema deve permettere a un *Organizer* di creare, modificare e cancellare i propri contest.
- **RF-02: Gestione Iscrizioni:** Il sistema deve permettere a un *Organizer* di invitare partecipanti e giurati tramite email. Gli inviti devono contenere token univoci.
- **RF-03: Sottomissione Lavori:** Il sistema deve permettere a un *Participant* di sottomettere un proprio *Work* (Lavoro Digitale) per un contest a cui è iscritto.
- **RF-04: Gestione Giurie e Form di Voto:** Il sistema deve permettere a un *Organizer* di creare giurie (di tipo *appointed* o *simple*) e di configurare un *VotingForm* associato. La configurazione del form deve permettere di definire campi organizzati secondo il loro **scope**: campi di *header* (mostrati una volta a inizio votazione), campi *participant* (ripetuti per ogni partecipante da valutare) e campi di *footer* (mostrati una volta a fine votazione). I tipi di campo supportati devono essere **testuali** (*textual*) e **slider**, per i quali deve essere possibile definire un intervallo di valori (min/max) compreso tra 0 e 10.
- **RF-05: Processo di Votazione:** Il sistema deve permettere a un utente autorizzato (sia *Appointed Juror* che *Simple Juror* tramite token) di inviare i propri voti durante una sessione live.
- **RF-06: Gestione Classifiche:** Il sistema deve consentire la generazione di una classifica basata sulla somma dei punteggi dei soli campi per i partecipanti. Deve permettere all'*Organizer* di pubblicare classifiche in formato pdf e scaricare i risultati “raw” in formato Excel.
- **RF-07: Notifiche Inbox:** Il sistema deve fornire a ogni utente autenticato una inbox per ricevere notifiche generate automaticamente da eventi di sistema.
- **RF-08: Amministrazione del Sistema:** Il sistema deve esporre un'interfaccia sicura (non per l'utente finale) per permettere a un *Admin* di eseguire operazioni di alto livello, come la visualizzazione di tutti gli utenti o la rimozione forzata di un contest.

2.4.2 Requisiti Non Funzionali

- **Usabilità (Usability):** L'interfaccia utente deve essere intuitiva, facile da apprendere e coerente tra la versione mobile e quella web.

- **Affidabilità (Reliability):** Il sistema deve essere robusto e garantire l'integrità dei dati. La logica di *snapshot* delle sessioni di voto è un requisito chiave per assicurare che il processo di valutazione sia immune da modifiche ai dati.
- **Performance:** L'applicazione deve essere reattiva, con tempi di caricamento inferiori ai 3 secondi. L'uso mirato del servizio Realtime è una scelta progettuale per minimizzare il traffico di rete.
- **Manutenibilità (Supportability):** L'architettura software deve essere modulare. L'adozione del pattern MVVM/BLoC per il frontend garantisce una netta separazione delle responsabilità, facilitando futuri interventi.
- **Implementazione (Implementation):**
 - Il client deve essere sviluppato in **Dart** con il framework **Flutter** per le piattaforme Android e Web.
 - Il backend deve basarsi sulla piattaforma **Supabase**, utilizzando **PostgreSQL**, **PL/pgSQL** per le RPC e **TypeScript** per le Edge Functions.
 - La gestione dello stato nel client deve seguire il pattern **BLoC**.
- **Interfaccia (Interface):**
 - Il sistema deve interfacciarsi in modo sicuro con servizi esterni come **Resend** (per le email) e **Google Places** (per la geolocalizzazione).
 - Deve essere prevista un'API sicura per l'interfacciamento con strumenti di amministrazione esterni (**Retool**).
- **Operativi (Operation):**
 - Il deployment del backend deve essere tracciabile tramite migrazioni SQL gestite dalla CLI di Supabase.
 - Il deployment del frontend web deve essere automatizzato tramite una pipeline di CI/CD.
 - I segreti (API keys) devono essere gestiti in modo sicuro, senza essere esposti nel codice sorgente.
- **Distribuzione (Packaging):**
 - L'applicazione web deve essere accessibile tramite URL pubblico.
 - L'applicazione Android deve essere distribuita come pacchetto **APK**, tramite un meccanismo di download sicuro.

- **Legali (Legal):**

- Il sistema deve essere sviluppato utilizzando principalmente tecnologie con licenze open-source permissive.
- La gestione dei dati personali deve essere orientata ai principi del GDPR, che garantisce all'utente il diritto alla cancellazione.

2.5 Modello del dominio

Il dominio applicativo è stato modellato a partire dalle entità fondamentali e dalle loro relazioni. Il Diagramma di Classe Concettuale (UML) è astratto dai dettagli implementativi e si focalizza sui concetti di business e sui vincoli che li governano.

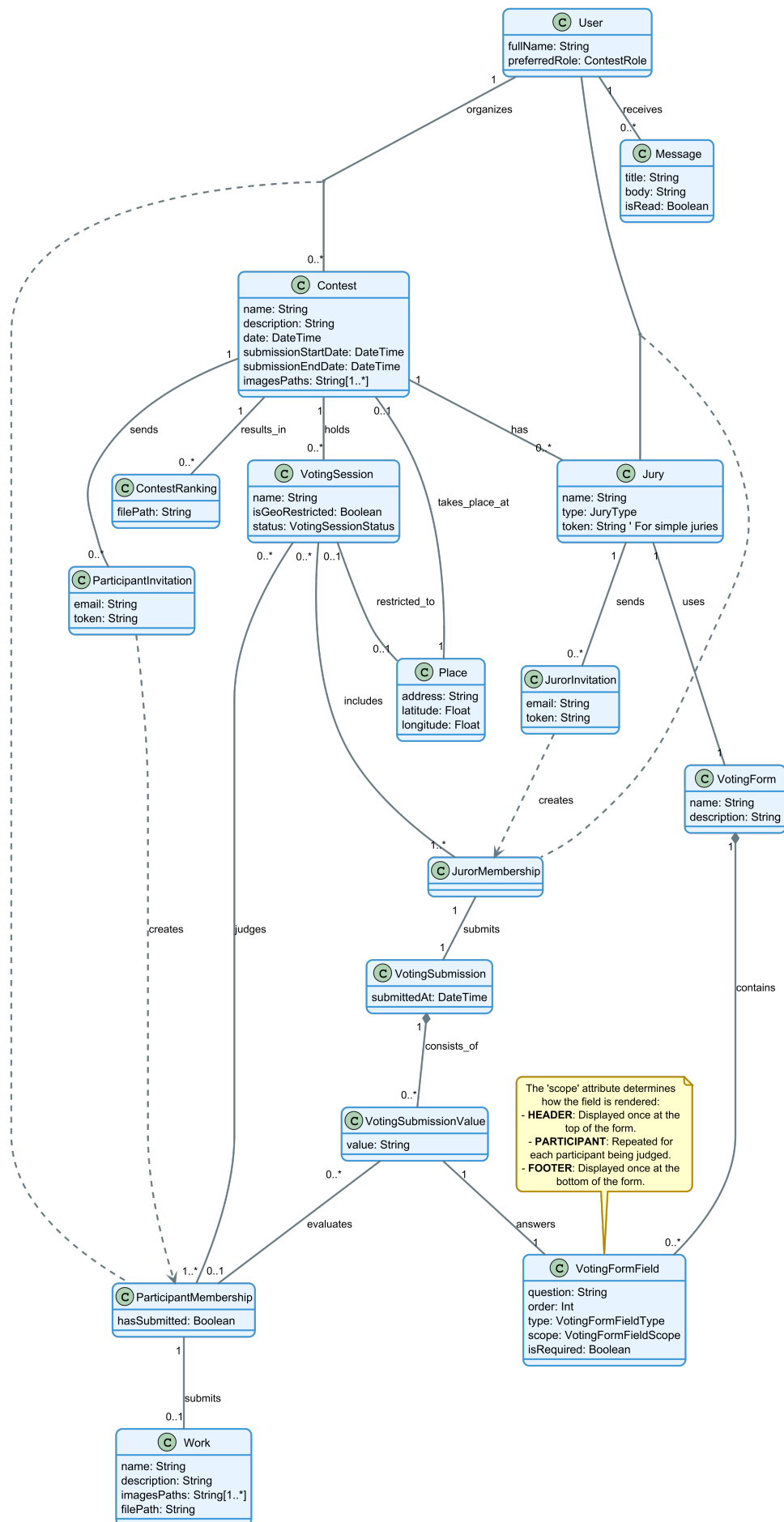


Figura 2.6: Diagramma delle classi

2.5.1 Entità Fondamentali

- **User:** l'utente del sistema, contenente informazioni del profilo.
- **Contest:** l'entità centrale, che aggrega tutte le informazioni relative a un concorso.
- **ParticipantMembership** e **JurorMembership:** classi associative che risolvono le relazioni “molti-a-molti” tra utenti e contest/giurie.
- **Work:** l'elaborato unico caricato da un partecipante. È legato a una *ParticipantMembership*.
- **Jury:** una giuria all'interno di un contest, che può essere di tipo *appointed* o *simple*.
- **VotingForm** e **VotingFormField:** la struttura del modulo di votazione. L'attributo *scope* del campo ne determina il comportamento.
- **VotingSession:** una sessione di voto live. Al suo avvio, il sistema “congela” lo stato del contest creando delle righe dedicate in tabelle di sessione (es. *voting_session_participants*), che contengono non solo i riferimenti alle entità originali, ma anche una copia di **attributi di snapshot** (es. il nome del partecipante, il nome dell'opera) per garantire l'immutabilità del processo.
- **ContestRanking:** la classifica finale, rappresentata come un riferimento a un file nello storage.
- **Message:** una notifica di sistema inviata a un utente.

2.5.2 Vincoli Chiave del Dominio

Il modello dati è stato progettato per far rispettare programmaticamente alcune regole di business fondamentali:

1. **Unicità del Work:** un partecipante può sottomettere un solo *Work* per ogni concorso a cui è iscritto, garantito da un vincolo di unicità a livello di database.
2. **Calcolo della Classifica Semplice:** la classifica viene generata esclusivamente sulla base dei campi di tipo *slider* con scope *participant*. I campi testuali non contribuiscono al punteggio.
3. **Immutabilità della Votazione:** il meccanismo di snapshot garantisce che una volta avviata una sessione di voto, i dati rilevanti dei partecipanti, dei giurati e la struttura del form vengano “congelati”. Questo avviene tramite la copia di attributi chiave in tabelle di sessione dedicate, rendendo il processo di valutazione immune da modifiche successive ai dati originali.

4. **Accesso Ospite Controllato:** l'accesso come *Simple Juror* è strettamente controllato da un token associato a una specifica giuria di tipo semplice (simple jury).

Capitolo 3

Tecnologie e Strumenti Utilizzati

La realizzazione del progetto **Swift Contest** è stata resa possibile dall'adozione di un insieme di tecnologie, piattaforme e strumenti moderni, selezionati per soddisfare i requisiti di cross-platform, sicurezza, scalabilità e velocità di sviluppo. Questo capitolo descrive lo stack tecnologico utilizzato, giustificando le scelte chiave in relazione agli obiettivi del progetto.

3.1 Stack Tecnologico Principale

Questa sezione descrive i linguaggi e i framework utilizzati per la scrittura del codice sorgente dell'applicazione, sia lato client che lato server.

3.1.1 Frontend: Flutter e Dart

- **Flutter:** È il framework UI open-source di Google scelto per lo sviluppo dell'applicazione client. La sua caratteristica principale è la capacità di generare applicazioni compilate nativamente per mobile, web e desktop da un'unica codebase. Questa scelta è stata fondamentale per soddisfare il requisito di **sviluppo cross-platform**, garantendo un'esperienza utente coerente e riducendo drasticamente i tempi e i costi di sviluppo e manutenzione.
- **Dart:** È il linguaggio di programmazione su cui si basa Flutter. È un linguaggio moderno, orientato agli oggetti e fortemente tipizzato, ottimizzato per la creazione di interfacce utente. La sua architettura, che supporta sia la compilazione Just-In-Time (JIT) per uno sviluppo rapido (con hot reload) sia la compilazione Ahead-Of-Time (AOT) per performance elevate in produzione, lo ha reso la scelta naturale per questo progetto.

3.1.2 Backend: Supabase, PL/pgSQL e TypeScript

- **Supabase:** È la piattaforma open-source **Backend-as-a-Service (BaaS)** che costituisce il cuore dell'infrastruttura server. È stata scelta perché offre, in un'unica soluzione integrata, tutti i servizi necessari: un database **PostgreSQL** completo, un sistema di autenticazione, uno storage per i file, API auto-generate e servizi serverless (RPC ed Edge Functions).
- **PL/pgSQL:** È il linguaggio procedurale di PostgreSQL. È stato utilizzato per scrivere tutte le **Funzioni RPC (Remote Procedure Call)** del sistema. La scelta di implementare la logica di accesso ai dati direttamente a livello di database tramite questo linguaggio ha permesso di ottenere performance elevate e di garantire l'atomicità delle operazioni, sfruttando al contempo il potente sistema di sicurezza di PostgreSQL.
- **TypeScript:** È stato scelto come linguaggio per lo sviluppo delle **Edge Functions**, eseguite sul runtime Deno di Supabase. La sua natura fortemente tipizzata, unita alla vasta disponibilità di librerie dell'ecosistema JavaScript/TypeScript, lo ha reso ideale per gestire logiche complesse, transazioni multi-step e, soprattutto, per orchestrare le interazioni sicure con le API di servizi esterni.

3.2 Piattaforme, Strumenti e Servizi

Questa sezione descrive le piattaforme di terze parti e gli strumenti che hanno supportato il ciclo di vita del progetto, dal development al deployment, fino alla documentazione.

3.2.1 Piattaforme di Deployment e Servizi Cloud

- **Vercel:** È la piattaforma serverless utilizzata per il deployment e l'hosting della versione **Web** dell'applicazione Flutter. È stata scelta per la sua integrazione nativa con i framework frontend moderni, la sua pipeline di CI/CD automatizzata e la sua capacità di gestire in modo ottimale le Single Page Applications (SPA) come Flutter Web.
- **Resend:** È il servizio transazionale di invio **email** utilizzato dal sistema. È stato integrato tramite la sua API REST, invocata da una Edge Function, per gestire l'invio di tutte le email automatiche, come gli inviti per partecipanti e giurati.
- **Google Cloud:** Sebbene il progetto non sia ospitato su Google Cloud, è stato utilizzato uno dei suoi servizi API: la **Google Places API**. Questo servizio è stato integrato per fornire la funzionalità di ricerca e autocompletamento degli indirizzi nella creazione dei contest e nella configurazione delle sessioni di voto per la geo-localizzazione.

3.2.2 Strumenti di Sviluppo e Amministrazione

- **IntelliJ IDEA:** È l'Integrated Development Environment (IDE) di JetBrains utilizzato per la scrittura del codice. Sebbene lo sviluppo iniziale fosse stato avviato su **Android Studio**, si è scelto di migrare a **IntelliJ IDEA Community Edition**. Questa decisione è stata motivata da una maggiore stabilità, una minore incidenza di bug e una superiore reattività dell'IDE, specialmente nel caricamento iniziale del progetto. Grazie alla possibilità di integrare il supporto completo per lo sviluppo in Flutter e Dart tramite plugin ufficiali, IntelliJ IDEA ha offerto un ambiente di sviluppo più performante e affidabile.
- **Material Theme Builder:** È uno strumento ufficiale di Google (disponibile online e su Figma) utilizzato per generare la base del tema Material Design 3. A partire da un singolo colore “seme” (seed color), lo strumento genera automaticamente l'intera palette di colori (ColorScheme) per il tema chiaro e scuro, garantendo coerenza cromatica e accessibilità. Il codice Dart generato da questo strumento è stato poi utilizzato come base per il MaterialTheme dell'applicazione.
- **Node Package Manager (npm):** È il gestore di pacchetti dell'ecosistema JavaScript/Node.js. Sebbene il progetto non sia basato su Node.js, npm è stato uno strumento indispensabile per l'installazione e la gestione di dipendenze di sviluppo, prima tra tutte la **CLI di Supabase**, utilizzata per gestire le migrazioni del database e il deployment delle Edge Functions.
- **Retool:** È la piattaforma low-code utilizzata per costruire rapidamente la **dashboard di amministrazione** interna. La sua capacità di interfacciarsi in modo nativo e sicuro con database PostgreSQL e API REST ha permesso di creare uno strumento di moderazione funzionale in poche ore, senza sottrarre tempo allo sviluppo dell'applicazione principale.

3.2.3 Version Control e Code Hosting

- **Git:** È il sistema di **controllo di versione distribuito** utilizzato per tracciare ogni modifica al codice sorgente del progetto. La sua adozione è stata fondamentale per gestire lo sviluppo in modo strutturato, permettendo la creazione di branch per nuove funzionalità, la gestione dei merge e la possibilità di tornare a versioni precedenti del codice.
- **GitHub:** È la piattaforma di **code hosting** che ospita il repository Git del progetto. Oltre al semplice hosting, è stata utilizzata per la sua integrazione con la pipeline di CI/CD di Vercel e, come descritto nel capitolo sulla distribuzione, per ospitare in modo sicuro le release dell'applicazione Android (APK).

3.2.4 Strumenti di Documentazione e Modellazione

- **PlantUML**: È lo strumento open-source utilizzato per creare tutti i diagrammi UML e architetturali. La sua caratteristica distintiva è l'approccio “diagrams as code”: invece di disegnare manualmente i diagrammi, questi vengono descritti attraverso un **linguaggio DSL (Domain-Specific Language)** testuale. Questa scelta ha permesso di creare diagrammi versionabili, facilmente modificabili e stilisticamente coerenti.

Capitolo 4

Progettazione e Implementazione del sistema

4.1 Linee guida progettuali

La progettazione di **Swift Contest** è stata guidata da principi di ingegneria del software moderni, volti a garantire un sistema robusto, sicuro e manutenibile, anche in un contesto di sviluppo rapido. Le linee guida principali sono state:

- **Separazione delle responsabilità (SoC)**: è stata applicata una netta separazione tra il livello di presentazione (UI), la logica di presentazione (ViewModel) e il livello di accesso ai dati (Model/Repository), fondamentale per la testabilità e la manutenibilità del codice.
- **Sicurezza come requisito primario**: è stato adottato un modello di accesso ai dati basato sul **Principio del Minimo Privilegio (Principle of Least Privilege)**, dove il client è considerato intrinsecamente non sicuro. L'accesso al database è negato per default e concesso selettivamente solo tramite un'API sicura.
- **Architettura Client-Server con Backend BaaS**: è stata adottata una moderna architettura Client-Server. Il client (sviluppato in Flutter) gestisce la presentazione e la logica di UI, mentre il server è rappresentato da una piattaforma **Backend-as-a-Service (BaaS)** come Supabase. Questo approccio permette di delegare la gestione dell'infrastruttura (database, autenticazione, storage) e di distribuire la logica di business in modo flessibile tra il client e le funzioni serverless (RPC e Edge Functions).
- **Sviluppo Cross-Platform**: la scelta di un framework come Flutter ha permesso di massimizzare il riuso del codice tra le piattaforme target (Android e Web), ottimizzando i tempi di sviluppo.

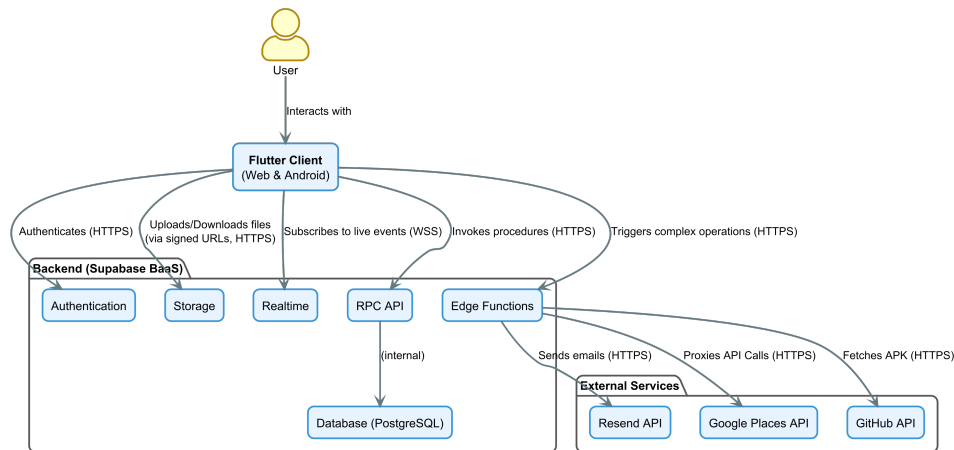


Figura 4.1: Architettura Client-Server del sistema Swift Contest con backend BaaS.

4.2 Architettura del frontend: il pattern MVVM con BLoC

Il frontend è stato sviluppato in Flutter seguendo i principi dell'architettura **MVVM (Model-View-ViewModel)**, implementata attraverso il pattern **BLoC (Business Logic Component)**. Questa scelta permette di ottenere una netta separazione delle responsabilità, garantendo testabilità e manutenibilità. La struttura del codice sorgente in `lib/` riflette questa suddivisione:

- **View (/view)**: il livello di presentazione, composto dai Widget di Flutter che costituiscono l'interfaccia utente. Questo livello si limita a mostrare i dati ricevuti dal suo View-Model (il BLoC) e a notificare gli eventi di interazione dell'utente (es. la pressione di un pulsante).
- **ViewModel (/viewmodel, implementato come BLoC)**: il cuore della logica di presentazione. Ogni BLoC riceve eventi dalla View, esegue la logica di business (spesso invocando un metodo del Model/Repository), e produce uno stream di stati a cui la View reagisce per aggiornarsi.
- **Model (/model)**: il livello di accesso ai dati. È composto dai *Repository*, che fungono da unica interfaccia verso il backend, astruendo le chiamate a RPC ed Edge Functions. Include anche le *Entities* (le rappresentazioni dei dati del dominio) e i *Bundles* (oggetti aggregati per ridurre il numero di chiamate di rete).

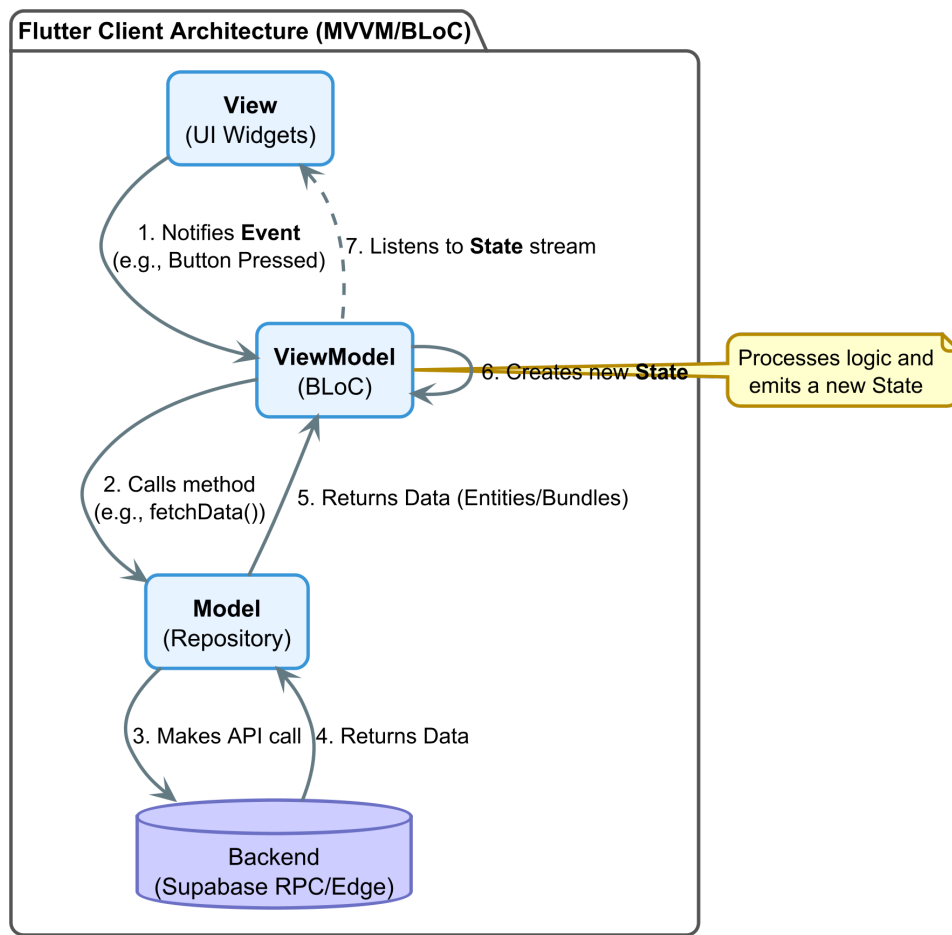


Figura 4.2: Architettura del Client MVVM con BLoC.

4.2.1 Implementazione del ViewModel: il pattern BLoC

Per la gestione dello stato e l'implementazione del ViewModel, è stato adottato il pattern BLoC, utilizzando il pacchetto `flutter_bloc`. La scelta di BLoC è motivata dalla sua natura reattiva e basata su stream, che si integra perfettamente con il paradigma dichiarativo di Flutter. Questo pattern impone un flusso di dati unidirezionale e prevedibile:

1. La **View** notifica un **Evento** (es. `LoginButtonPressed`).
2. Il **BLoC** riceve l'evento, esegue la logica di business (es. chiama `authRepository.login()`) e, al termine, emette un nuovo **Stato** (es. `LoginSuccess` o `LoginFailure`).
3. La **View**, tramite un widget come `BlocBuilder`, si mette in ascolto dello stream di stati e si ridisegna per riflettere il nuovo stato.

Questo approccio garantisce che la logica di business sia completamente disaccoppiata dalla UI, rendendola facilmente testabile in isolamento. I BLoC sono stati organizzati in due categorie: **globali** (es. `AuthBloc`, `InboxBloc`) e **di pagina** (es. `JurorVotingProcedurePageBloc`).

Gestione Centralizzata degli Errori nel Repository

Una delle responsabilità principali del livello *Model* (e in particolare dei *Repository*) è quella di agire come un robusto intermediario tra la logica di business e il backend. Questo include non solo il recupero dei dati, ma anche una **gestione centralizzata e sicura degli errori**.

Le chiamate a un backend come Supabase possono fallire in molti modi: problemi di rete (`SocketException`), errori di autenticazione (`AuthException`), errori del database (`PostgreSQLException`), ecc. Esporre queste eccezioni direttamente al livello del BLoC o della UI sarebbe problematico per due motivi:

1. Creerebbe un forte accoppiamento tra la logica di presentazione e le librerie specifiche del backend.
2. Potrebbe esporre all'utente finale messaggi di errore tecnici e potenzialmente sensibili.

Per risolvere questo problema, è stata creata una funzione di ordine superiore, `handleBackendCall`, che "avvolge" ogni chiamata al backend. Come mostrato nel listato seguente, questa funzione utilizza un blocco `try...catch` per intercettare le eccezioni specifiche di Supabase e le mappa in classi di **Failure** personalizzate e sicure. Queste *Failure* sono oggetti semplici che rappresentano un fallimento in modo astratto (es. `NetworkFailure`, `ServerFailure`), senza contenere dettagli tecnici. Il tipo di ritorno `Either<Failure, T>` permette al BLoC di gestire in modo pulito e tipizzato sia il caso di successo (a destra) che quello di fallimento (a sinistra).

```
1  /// Avvolge una chiamata al backend, gestendo e mappando le eccezioni.
2  Future<Either<Failure, T>> handleBackendCall<T>(  
3  Future<Either<Failure, T>> Function() function,  
4  ) async {  
5    try {  
6      // Esegue la funzione che contiene la chiamata al backend.  
7      return await function();  
8    } on SocketException catch (e) {  
9      // Errore di rete (es. no internet).  
10     Logger.warning('Network error: $e');  
11     return Either.left(const NetworkFailure());  
12   } on AuthException catch (e) {  
13     // Errore di autenticazione di Supabase.  
14     Logger.warning('Auth error: $e');  
15     return Either.left(authExceptionToFailure(e));  
16   } on PostgreSQLException catch (e) {  
17     // Errore del database PostgreSQL.  
18     Logger.warning('Postgrester error: $e');  
19     return Either.left(postgresterExceptionToFailure(e));  
20   } on FunctionException catch (e) {  
21     // Errore durante l'invocazione di una Edge Function.  
22     Logger.warning('Edge function error: $e');
```

```

23     return Either.left(edgeFunctionExceptionToFailure(e));
24   } on StorageException catch (e) {
25     // Errore nello Storage (es. upload fallito).
26     Logger.warning('Storage error: $e');
27     return Either.left(storageExceptionToFailure(e));
28   } catch (e) {
29     // Qualsiasi altro errore non previsto.
30     Logger.error('Unknown error: $e');
31     return Either.left(const Failure());
32   }
33 }

```

Listing 4.1: Funzione helper per la gestione sicura delle chiamate al backend.

Gestione dell’Uguaglianza degli Oggetti con Equatable

Per ottimizzare le performance e garantire un comportamento prevedibile nella gestione dello stato, tutte le entità, i bundle e gli stati dei BLoC estendono la classe fornita dal pacchetto `equatable`.

Questa scelta implementativa è cruciale per due motivi principali.

Il primo riguarda il corretto funzionamento del pattern BLoC. Di default, Dart confronta gli oggetti per riferimento, non per valore. Ciò significa che due istanze di un oggetto, anche se contengono dati identici, vengono considerate diverse. Senza `Equatable`, un BLoC potrebbe emettere un nuovo stato identico al precedente, causando una ricostruzione non necessaria e dispendiosa dell’interfaccia utente. Utilizzando `Equatable`, invece, il confronto avviene basandosi sul valore degli attributi dell’oggetto. Questo permette ai widget di Flutter, come `BlocBuilder`, di determinare in modo efficiente se uno stato è *realmente* cambiato prima di procedere con la ricostruzione, prevenendo refresh superflui della UI e migliorando significativamente la reattività dell’applicazione.

Il secondo motivo, altrettanto importante, riguarda la **robustezza e la chiarezza della logica di business**. Spesso, durante lo sviluppo, sorge la necessità di confrontare due oggetti per verificare se rappresentano la stessa entità concettuale, anche se sono istanze diverse in memoria. Estendendo `Equatable`, si ottiene un’implementazione standard e priva di errori dell’uguaglianza basata sul valore, semplificando la logica di confronto all’interno dei BLoC e dei Repository e riducendo il rischio di bug sottili legati al confronto per riferimento.

4.2.2 Navigazione con AutoRoute

La gestione della navigazione è stata affidata al pacchetto `auto_route`, che genera un router fortemente tipizzato, eliminando l’uso di stringhe per le rotte e riducendo il rischio di errori a runtime. Un elemento chiave è l’`AuthGuard`, una classe che intercetta le richieste di navigazione

e verifica lo stato di autenticazione dell'utente tramite l'AuthBloc, reindirizzandolo alla pagina di login se tenta di accedere a una rotta protetta.

```
1 // --- 1. Definizione della rotta protetta (in app_router.dart) ---
2
3 @AutoRouterConfig()
4 class AppRouter extends RootStackRouter {
5   // ...
6   @override
7   List<AutoRoute> get routes => [
8
9     // ... altre rotte pubbliche o di autenticazione
10
11     // Esempio di rotta protetta: solo gli utenti autenticati
12     // possono accedere alla dashboard dell'organizzatore.
13     AutoRoute(
14       path: '/organizer-home',
15       page: OrganizerHomeRoute.page,
16       guards: [AuthGuard(authBloc: authBloc)], // Applica la guardia
17     ),
18
19     // ... altre rotte protette
20   ];
21 }
22
23
24 // --- 2. Implementazione della guardia (in auth_guard.dart) ---
25
26 class AuthGuard extends AutoRouteGuard {
27   final AuthBloc authBloc;
28
29   AuthGuard({required this.authBloc});
30
31   @override
32   void onNavigation(NavigationResolver resolver, StackRouter router) {
33     // Controlla lo stato di autenticazione tramite l'AuthBloc
34     final isAuthenticated = authBloc.state.authStatus.isAuthenticated;
35
36     // Elenco completo delle rotte pubbliche (di autenticazione)
37     const authPaths = [
38       '/sign-in',
39       '/sign-in-verify',
40       '/sign-up',
41       '/sign-up-verify'
42     ];
43     final targetPath = resolver.route.path.split('/:').first;
44
```

```

45     if (isAuthenticated) {
46         // Se l'utente è autenticato e cerca di andare a una pagina di
login/registrazione,
47         // lo reindirizzo alla sua home page.
48         if (authPaths.contains(targetPath)) {
49             resolver.redirect(const RootRoute());
50         } else {
51             // Altrimenti, può procedere verso la rotta protetta.
52             resolver.next(true);
53         }
54     } else {
55         // Se l'utente NON è autenticato...
56         if (!authPaths.contains(targetPath)) {
57             // ...e sta cercando di accedere a una rotta protetta,
58             // lo reindirizzo alla pagina di login.
59             resolver.redirect(SignInRoute());
60         } else {
61             // ...e sta già andando a una pagina di login/registrazione, lo
lascio procedere.
62             resolver.next(true);
63         }
64     }
65 }
66 }

```

Listing 4.2: Estratto del sistema di routing protetto con AuthGuard.

4.2.3 Strategia di Navigazione e Passaggio dei Dati

Una scelta architetturale fondamentale, dettata dalla necessità di una perfetta integrazione con il web, riguarda la modalità di passaggio dei dati tra le pagine. A differenza di un'applicazione puramente mobile, dove è comune passare interi oggetti in memoria durante la navigazione, in un'applicazione web questo approccio è insostenibile: l'utente può ricaricare la pagina o accedere a un URL diretto, perdendo qualsiasi stato tenuto in memoria.

Per questo motivo, il sistema di navigazione di Swift Contest adotta una strategia rigorosa: **nelle rotte vengono passati solo identificativi (ID) primitivi** (es. `contestId`, `juryId`), mai oggetti complessi. Quando una pagina viene caricata, il BLoC associato riceve l'ID dai parametri della rotta e, come prima azione, invoca il Repository per caricare dal backend tutti i dati necessari.

Questa scelta, sebbene aumenti il numero di chiamate al database, garantisce enormi vantaggi in termini di robustezza e coerenza con il paradigma web:

- Ogni pagina è **ricaricabile** in modo indipendente.
- Ogni pagina ha un **URL univoco e condivisibile** (deep linking).

- Il comportamento dell'applicazione è prevedibile e consistente tra la navigazione interna e l'accesso diretto a un URL.

Validazione dell'Input Utente e Gestione dei Form

Per garantire l'integrità dei dati e fornire un feedback immediato all'utente, è stata implementata una robusta strategia di validazione dell'input direttamente a livello client, prima che i dati vengano inviati al BLoC e, successivamente, al backend.

Invece di definire la logica di validazione all'interno dei singoli widget, **tutte le funzioni di validazione sono state centralizzate** in un unico file, `lib/utils/validators/validators.dart`. Questo approccio, basato su funzioni pure, offre tre vantaggi significativi:

- **Riutilizzabilità:** La stessa funzione, ad esempio `emailValidator`, può essere riutilizzata in più punti dell'applicazione (es. pagina di login, registrazione, modifica profilo) senza duplicazione di codice.
- **Manutenibilità:** Se una regola di validazione deve essere modificata (es. la complessità minima di una password), la modifica viene effettuata in un unico punto, garantendo coerenza in tutta l'applicazione.
- **Testabilità:** Essendo funzioni pure che accettano una stringa e restituiscono un risultato, i validatori sono estremamente facili da testare in isolamento con unit test, senza la necessità di costruire alcun widget dell'interfaccia utente.

Il listato seguente mostra un esempio di un validatore complesso e il suo utilizzo all'interno di un widget `TextFormField`.

```

1  // 1. Definizione della funzione di validazione pura
2  // in lib/utils/validators/validators.dart
3
4  String? passwordValidator(String? value) {
5    if (value == null || value.isEmpty) {
6      return 'Required';
7    }
8    if (value.length < 8) {
9      return 'Must contain at least 8 characters';
10   }
11   if (!RegExp(r'[A-Z]').hasMatch(value)) {
12     return 'Must contain at least one uppercase letter';
13   }
14   if (!RegExp(r'[0-9]').hasMatch(value)) {
15     return 'Must contain at least one number';
16   }
17   if (!RegExp(r'[@#$%*~%^() \[\] \- _ = + { } ; : , . < > ? / \ | ]').hasMatch(value)) {
18     return 'Must contain at least one special character';

```

```

19     }
20
21     return null; // La validazione ha successo
22 }
23
24 // 2. Utilizzo del validatore in un widget della UI
25
26 TextFormField(
27   // ... altre proprietà come controller, decoration, etc.
28
29   // La funzione di validazione viene passata come riferimento.
30   validator: passwordValidator,
31
32   // La validazione viene attivata automaticamente al momento
33   // del submit del form o in base all'autovalidateMode.
34   autovalidateMode: AutovalidateMode.onUserInteraction,
35 );

```

Listing 4.3: Esempio di validatore centralizzato e suo utilizzo in un widget.

4.2.4 Design System e Theming con Estensioni

Per garantire un'interfaccia utente coerente e facilmente manutenibile, è stato definito un design system centralizzato basato sul **Material Design 3**. La scelta di `MaterialTheme` come base è stata strategica, in quanto è lo standard de facto per le applicazioni Android e garantisce un'esperienza utente nativa e riconoscibile.

Tuttavia, il `ColorScheme` standard di Material (con colori come `primary`, `secondary`, `surface`, ecc.) non è sufficiente per coprire tutte le esigenze di un'applicazione complessa, che richiede colori personalizzati.

Per risolvere questo problema in modo pulito e scalabile, invece di definire colori statici sparsi nel codice, è stata creata un'**estensione Dart** sulla classe `ColorScheme`. Questa tecnica permette di "arricchire" il tema esistente con nuovi colori personalizzati.

Questi colori personalizzati diventano così parte integrante del tema e possono essere richiamati in qualsiasi punto dell'applicazione in modo sicuro e coerente tramite `Theme.of(context).colorScheme`, su ereditando automaticamente i cambiamenti tra tema chiaro e scuro.

```

1 // in lib/utils/themes/color_scheme_x.dart
2
3 extension ColorSchemeX on ColorScheme {
4   // Colore per indicare successo (es. notifiche positive)
5   Color get success => const Color(0xFF28a745);
6   Color get onSuccess => const Color(0xFFFFFFFF);
7

```

```

8      // Colore per avvisi
9      Color get warning => const Color(0xFFffc107);
10     Color get onWarning => const Color(0xFF212529);
11
12     // Altri colori personalizzati...
13 }

```

Listing 4.4: Definizione dell'estensione ColorSchemeX per aggiungere colori personalizzati al tema.

Gestione del Feedback di Caricamento: l'OverlayLoader

Per fornire un feedback visivo all'utente durante le operazioni asincrone, è stato implementato un sistema di caricamento personalizzato, OverlayLoader. Questo sistema utilizza l'Overlay di Flutter per mostrare un indicatore di progresso con uno sfondo semitrasparente e sfocato, permettendo all'utente di mantenere il contesto visivo della schermata sottostante.

L'implementazione si basa su due scelte di design chiave per garantire robustezza e facilità d'uso:

- **Singleton Pattern:** La classe OverlayLoader è implementata come un singleton per garantire che esista una sola istanza del gestore del loader, prevenendo la visualizzazione di più overlay contemporaneamente.
- **Dart Extensions:** Per semplificare l'invocazione del loader, è stata creata un'estensione sulla classe BuildContext, che permette di usare una sintassi pulita e intuitiva come `context.showLoader()` e `context.hideLoader()`.

Gestione del Ciclo di Vita e Prevenzione dei "Leak" Un aspetto cruciale per la robustezza di questo sistema è la gestione del suo ciclo di vita. Per evitare che l'overlay del loader rimanga "bloccato" sullo schermo se una pagina viene chiusa (*disposed*) durante un'operazione di caricamento (ad esempio, a causa di un errore o della navigazione dell'utente), è stata adottata una pratica difensiva: **il metodo `context.hideLoader()` viene sistematicamente invocato all'interno del metodo `dispose` di ogni pagina** che utilizza il loader. Questo garantisce che, indipendentemente da come o perché la pagina viene distrutta, qualsiasi overlay di caricamento attivo venga sempre rimosso, prevenendo "leak" visuali e garantendo la stabilità dell'interfaccia utente.

```

1      // 1. Classe Singleton per la gestione dell'Overlay
2      class OverlayLoader {
3          OverlayEntry? _entry;
4
5          // Singleton pattern per garantire un'unica istanza
6          OverlayLoader._();

```

```

7   static final OverlayLoader _instance = OverlayLoader._();
8   factory OverlayLoader() => _instance;
9
10  void show(BuildContext context) {
11    if (_entry != null) return; // Evita di mostrare più loader
12
13    _entry = OverlayEntry(
14      builder: (_) => const Positioned.fill(
15        // ObscuredLoader è un widget custom con sfondo sfocato
16        child: ObscuredLoader(),
17      ),
18    );
19
20    // Inserisce l'entry nell'overlay radice per essere sempre in cima
21    Overlay.of(context, rootOverlay: true).insert(_entry!);
22  }
23
24  void hide() {
25    _entry?.remove();
26    _entry = null;
27  }
28 }
29
30 // 2. Estensione su BuildContext per un'API pulita e intuitiva
31 extension OverlayLoaderX on BuildContext {
32   void showLoader() => OverlayLoader().show(this);
33   void hideLoader() => OverlayLoader().hide();
34 }
35
36 // 3. Utilizzo difensivo nel ciclo di vita di un widget di pagina
37 class MyPage extends StatefulWidget { // ... }
38
39   class _MyPageState extends State<MyPage> {
40     // ...
41
42     @override
43     void dispose() {
44       // Garantisce che il loader venga sempre rimosso quando la pagina
45       // viene distrutta, prevenendo "leak" visuali.
46       context.hideLoader();
47       super.dispose();
48     }
49
50     // ...
51   }

```

Listing 4.5: Implementazione del loader non-intrusivo tramite Overlay e Dart Extensions.

4.3 Progettazione del Database

Il modello concettuale del dominio, descritto nel Capitolo 2, è stato tradotto in un modello logico e fisico implementato su un database **PostgreSQL**, gestito tramite la piattaforma Supabase. La progettazione si è concentrata su tre pilastri: integrità dei dati, sicurezza e performance.

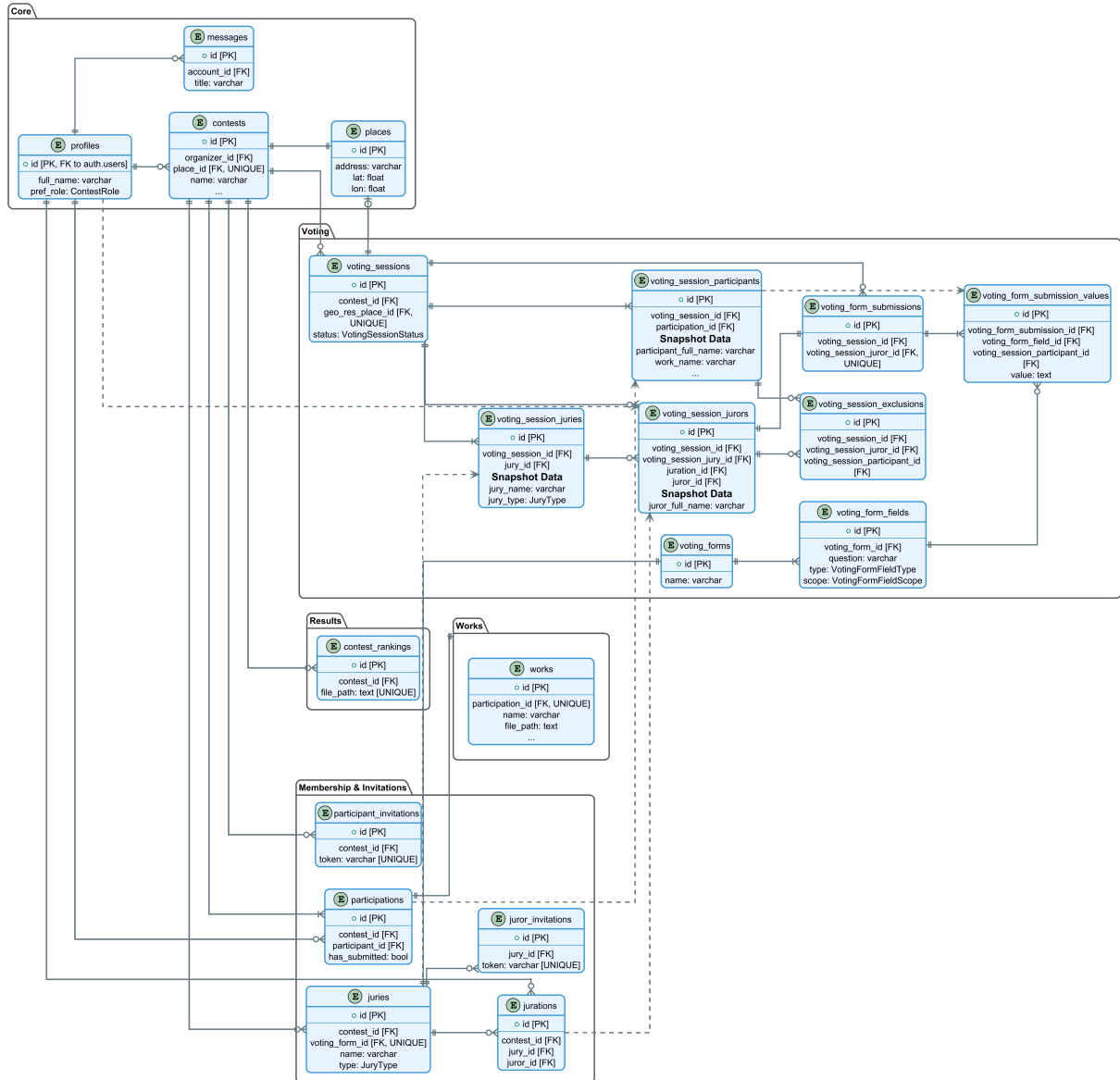


Figura 4.3: Diagramma Entità-Relazioni (ER) dello schema del database.

4.3.1 Schema Fisico e Tabelle Principali

Lo schema del database è composto da una serie di tabelle che implementano le entità del dominio. Le tabelle cardine includono:

- **profiles:** Questa tabella estende la tabella `auth.users` gestita internamente da Supabase. Poiché lo schema `auth` è protetto e non modificabile, per aggiungere attributi per-

sonalizzati (es. `full_name`, `pref_role`) è stata creata una tabella parallela nello schema `public`. La colonna `id` di `profiles` funge sia da chiave primaria che da chiave esterna verso `auth.users(id)`, stabilendo una relazione uno-a-uno. La creazione del profilo è automatizzata tramite un **trigger** a livello di database, che inserisce un nuovo record in `profiles` ogni volta che un utente completa la registrazione.

- `contests`: memorizza le informazioni principali di un concorso.
- `participations` e `jurations`: implementano le classi di associazione che legano utenti, contest e giurie.
- `works`: contiene i metadati delle opere sottomesse dai partecipanti.
- `voting_forms` e `voting_form_fields`: definiscono la struttura dei moduli di voto.
- `voting_sessions`: rappresenta una sessione di voto.
- `voting_session_participants`, `voting_session_jurors`: contengono gli snapshot dei dati per una sessione di voto attiva.
- `voting_form_submissions` e `voting_form_submission_values`: memorizzano i voti inviati dai giurati.

4.3.2 Scelte di Design Chiave

Oltre alla semplice mappatura delle entità, sono state prese decisioni di design cruciali per garantire la robustezza del sistema.

Integrità Referenziale e Strategia di Cancellazione

Per garantire la consistenza del database, è stato fatto un uso estensivo dei vincoli di chiave esterna. Una decisione di design fondamentale è stata quella di adottare una strategia di **cancellazione fisica (hard delete)** piuttosto che una di **cancellazione logica (soft delete)**, principalmente per ottimizzare l'uso delle risorse (spazio su database e storage) e per semplificare la logica delle query di lettura.

Data questa scelta, la gestione della cancellazione dei dati correlati è diventata un aspetto critico, risolto con un approccio a più livelli.

Cancellazione a Cascata tramite ON DELETE CASCADE Per le relazioni di composizione forte e dirette, dove l'entità "figlia" non ha dipendenze esterne come file nello Storage, è stata utilizzata la politica **ON DELETE CASCADE**. L'esempio più calzante è la relazione tra `voting_forms` e `voting_form_fields`. Poiché i campi di un form non hanno senso di esistere senza il form stesso, l'eliminazione di un record in `voting_forms` provoca la cancellazione automatica e a

cascata di tutti i record correlati in `voting_form_fields`. Questo approccio delega interamente la logica di pulizia al database, garantendo massima efficienza e consistenza dei dati con il minimo sforzo implementativo.

Cancellazione tramite Trigger per Relazioni Uno-a-Uno In alcune relazioni uno-a-uno, la chiave esterna è stata posizionata nella tabella concettualmente "principale" per motivi di design. L'esempio più significativo è la relazione tra `contests` e `places`: è la tabella `contests` a contenere il riferimento `place_id` con un vincolo `UNIQUE`.

Questa configurazione, sebbene logicamente corretta, impedisce di usare la politica `ON DELETE CASCADE` per eliminare un `place` quando il `contest` associato viene cancellato. Per risolvere questo problema e automatizzare la pulizia dei dati, è stato implementato un **trigger di database**. Questo trigger si attiva *dopo* (AFTER) l'eliminazione di un record dalla tabella `contests` ed esegue esplicitamente la cancellazione del record corrispondente nella tabella `places`, utilizzando l'ID del luogo del contest appena eliminato. In questo modo, si garantisce l'integrità referenziale e si evita di lasciare record `places` orfani, delegando anche in questo caso la logica di pulizia al database.

Cancellazione di Dati Esterni tramite Edge Functions La cancellazione a cascata del database non può gestire dati esterni, come i file memorizzati in Supabase Storage. Per questo motivo, le operazioni di cancellazione più complesse, come l'eliminazione di un utente o di un contest, non vengono eseguite direttamente dal client, ma sono gestite da **Edge Functions** dedicate (es. `user-delete-account`, `admin-delete-contest`). Queste funzioni agiscono da orchestratori: dopo aver eliminato i record dal database (che a loro volta attivano le cancellazioni a cascata e i trigger), invocano l'API di Supabase Storage per **eliminare tutti i file associati** (es. immagini del profilo, file dei *Work*), garantendo una pulizia completa e prevenendo la creazione di file "orfani". Questa logica non poteva essere implementata in una funzione RPC, poiché queste non hanno accesso all'API dello Storage.

Snapshot dei Dati tramite Denormalizzazione Controllata

Per garantire l'immutabilità e l'auditabilità del processo di voto, è stato implementato un meccanismo di **snapshot dei dati**, una delle scelte architetturali più importanti del backend.

Quando un organizzatore avvia una *VotingSession* tramite la funzione RPC `organizer_start_voting_session`, il sistema non si limita a collegare le entità esistenti, ma esegue una forma di **denormalizzazione controllata**. Per ogni partecipante e giurato coinvolto, vengono create nuove righe in tabelle di sessione dedicate (es. `voting_session_participants`, `voting_session_jurors`).

Queste nuove righe contengono non solo le chiavi esterne ai record originali, ma anche una copia statica degli attributi più importanti al momento dell'avvio. Come mostrato nel listato seguente,

la tabella `voting_session_participants` include, oltre a `participation_id`, anche i campi `participant_full_name`, `work_name`, ecc.

Una scelta di design cruciale è l'uso della clausola `ON DELETE SET NULL` sulla chiave esterna `participation_id`. Questo significa che se il partecipante originale dovesse essere eliminato dal sistema, il record dello snapshot *non* verrebbe cancellato, ma manterrebbe solo il riferimento a NULL. Ciò dimostra che la vera "fonte di verità" per la sessione di voto sono gli attributi di snapshot, garantendo che i voti espressi rimangano validi e auditabili anche se le entità originali cessano di esistere.

Lo stesso principio si applica a `voting_session_jurors`, dove `juror_id` è fondamentale per tracciare se un giurato ha già votato, ma può essere impostato a NULL in caso di cancellazione dell'account senza invalidare il voto già espresso.

Questa tecnica garantisce:

1. **Integrità della sessione:** se un utente cambia nome, il nome "congelato" nello snapshot rimane invariato.
2. **Auditabilità e resilienza:** i voti rimangono validi e associati ai dati di snapshot anche se i record originali vengono eliminati.
3. **Performance ottimizzate:** i dati denormalizzati vengono letti direttamente, evitando complesse query con join.

```
1  -- Definizione della tabella `voting_session_participants`
2
3  CREATE TABLE public.voting_session_participants (
4    id uuid PRIMARY KEY DEFAULT extensions.uuid_generate_v4(),
5    voting_session_id uuid NOT NULL REFERENCES public.voting_sessions(id) ON
      DELETE CASCADE,
6
7    -- Riferimento opzionale al record di partecipazione originale.
8    -- ON DELETE SET NULL: se il partecipante viene rimosso dal contest,
9    -- lo snapshot sopravvive, garantendo l'integrità dei voti.
10   participation_id uuid REFERENCES public.participations(id) ON DELETE SET
      NULL,
11
12   order_index int NOT NULL,
13
14   -- --- Attributi di Snapshot ---
15   -- Dati "congelati" al momento dell'avvio della sessione.
16   participant_full_name varchar(70) NOT NULL,
17   work_name varchar(50) NOT NULL,
18   work_description varchar(1000) NOT NULL,
19   work_images_paths text[] NOT NULL,
```

```

20
21 UNIQUE (voting_session_id, participation_id),
22 UNIQUE (voting_session_id, order_index)
23 );
24
25 -- Definizione della tabella `voting_session_jurors`
26
27 CREATE TABLE public.voting_session_jurors (
28 id uuid PRIMARY KEY DEFAULT extensions.uuid_generate_v4(),
29 voting_session_id uuid NOT NULL REFERENCES public.voting_sessions(id) ON
    DELETE CASCADE,
30 voting_session_jury_id uuid NOT NULL REFERENCES public.voting_session_
    juries(id) ON DELETE CASCADE,
31
32 -- Riferimento opzionale al giurato.
33 -- ON DELETE SET NULL: se l'account del giurato viene eliminato,
34 -- il suo voto rimane registrato ma anonimizzato.
35 juror_id uuid REFERENCES public.profiles(id) ON DELETE SET NULL,
36
37 has_submitted bool NOT NULL DEFAULT false,
38
39 -- --- Attributo di Snapshot ---
40 juror_full_name varchar(70) NOT NULL,
41
42 UNIQUE (voting_session_jury_id, juror_id)
43 );

```

Listing 4.6: Definizione delle tabelle di sessione con attributi di snapshot e gestione della cancellazione.

4.3.3 Implicazioni del Design del Form di Voto sull'Aggregazione dei Risultati

Una scelta di design fondamentale, visibile nello schema del database, è che **ogni giuria è associata a un unico e specifico form di voto**. Questa flessibilità permette all'organizzatore di creare criteri di valutazione diversi per giurie diverse (es. una giuria tecnica con campi slider complessi e una giuria popolare con un singolo campo di preferenza).

Tuttavia, questa decisione architetturale ha un'implicazione diretta e importante su come i risultati possono essere processati e visualizzati, influenzando sia il backend che il frontend: **il sistema non può aggregare automaticamente i punteggi tra giurie diverse**, poiché i loro form di voto (e quindi i loro criteri e punteggi) non sono direttamente comparabili.

Di conseguenza, tutte le operazioni di analisi dei risultati, sia a livello di logica di backend (RPC) che di interfaccia utente (frontend), sono state progettate per operare a livello della singola giuria:

- La **generazione della classifica** avviene per ogni singola giuria.
- L'**esportazione dei risultati** in formato Excel è granulare e produce un file per ciascuna giuria.

Se l'organizzatore desidera creare una classifica "globale" che combini i risultati di più giurie, deve farlo esternamente, utilizzando i dati esportati. Questa è una limitazione consapevole del progetto, un compromesso accettato per garantire la massima flessibilità nella configurazione dei criteri di valutazione.

Per mantenere questa flessibilità anche nella fase finale, il sistema non impone un formato di classifica globale, ma permette all'organizzatore di **pubblicare qualsiasi classifica in formato PDF**. Questa scelta di design gli conferisce la **massima libertà**: può decidere di pubblicare la classifica di una singola giuria, oppure può elaborare esternamente una classifica globale personalizzata e caricarla come file PDF. In entrambi i casi, una volta pubblicata, la classifica diventa disponibile per il download a tutti i partecipanti e giurati del contest all'interno dell'applicazione.

4.4 Architettura del backend

Il backend è costruito interamente su Supabase e la sua architettura è definita da un'API sicura composta da un insieme di Funzioni RPC e Edge Functions.

4.4.1 Funzioni RPC (Remote Procedure Call)

Le funzioni RPC, scritte in PL/pgSQL, sono il principale punto di ingresso per le operazioni sul database. Sono tutte dichiarate con SECURITY DEFINER, il che significa che vengono eseguite con i privilegi del loro creatore (un ruolo amministrativo) e non con quelli dell'utente chiamante. Al loro interno, contengono logica di validazione e controllo dei permessi, tipicamente verificando l'identità dell'utente tramite `auth.uid()`. Questa scelta è il fulcro dell'approccio di **accesso mediato da API**: nessun utente può effettuare operazioni dirette sul database, ma deve passare attraverso questo strato di controllo che garantisce che non possa compiere azioni a lui non concesse.

Nome Funzione	Scopo Principale	Controllo di Sicurezza Esempio
<code>auth_get_account_bundle</code>	Ottiene i dati completi dell'utente autenticato.	<code>auth.uid() = id</code>
<code>organizer_create_contest</code>	Crea un nuovo contest e il luogo associato.	Verifica che l'utente sia autenticato.

Nome Funzione	Scopo Principale	Controllo di Sicurezza Esempio
organizer_start_voting_session	Avvia una sessione di voto, creando lo snapshot dei dati.	L'utente deve essere l'organizzatore del contest.
participant_join_contest	Associa un partecipante a un contest tramite un token di invito.	Il token deve essere valido e non utilizzato.
juror_submit_votes	Salva i voti di un giurato per una sessione.	La sessione deve essere 'live' e il giurato non deve aver già votato.
juror_access_voting_as_simple_juror	Fornisce l'accesso a un giurato ospite tramite il token della giuria.	Il token della giuria deve essere valido.

Tabella 4.1: Elenco esemplificativo delle Funzioni RPC del sistema.

```

1  -- STEP 2.5: VALIDATE VOTE VALUES
2  -- Controlla in modo efficiente se qualche valore di tipo 'slider' nel
   payload JSON
3  -- è al di fuori del suo range min/max definito o non è un numero valido.
4  IF EXISTS (
5  -- Utilizza una Common Table Expression (CTE) per "spacchettare" il
   payload JSON
6  -- in una tabella temporanea di righe e colonne.
7  WITH unpacked_votes AS (
8  SELECT
9  (value->>'voting_form_field_id')::uuid as field_id,
10 (value->>'value')::text as value
11 FROM jsonb_array_elements(p_votes_payload)
12 )
13 -- Seleziona le righe che violano le regole di validazione.
14 SELECT 1
15 FROM unpacked_votes uv
16 -- Unisce i voti spacchettati con la tabella dei campi del form
17 -- per ottenere i metadati di ogni campo (tipo, min, max).
18 JOIN public.voting_form_fields vff ON vff.id = uv.field_id
19 WHERE
20 -- Applica il controllo solo ai campi di tipo 'slider'.
21 vff.type = 'slider'
22 AND (
23 -- Condizione 1: Il valore non è una stringa di un numero intero.
24 NOT (uv.value ~ '^-\d+$')

```

```

25  -- Condizione 2: Oppure, se è un intero, è fuori dai limiti.
26  OR uv.value::integer < vff.slider_min_value
27  OR uv.value::integer > vff.slider_max_value
28  )
29  ) THEN
30  -- Se anche solo una riga viola le condizioni, solleva un'eccezione
31  -- e interrompe la transazione, impedendo l'inserimento dei dati.
32  RAISE EXCEPTION 'Invalid vote value: A slider value is outside its
    allowed range or is not a valid number.';
33  END IF;
34
35  -- Se la validazione passa, la funzione procede con l'inserimento dei
    voti...

```

Listing 4.7: Estratto dalla funzione `juror_submit_votes` che mostra la validazione server-side dei valori degli slider.

4.4.2 Edge Functions

Le Edge Functions, scritte in TypeScript ed eseguite su Deno, gestiscono operazioni complesse, transazioni multi-step e integrazioni con servizi esterni che non sarebbero possibili o sicure da eseguire in una semplice RPC.

Nome Funzione	Scopo	Autorizzazione / Dipendenze
organizer-invite-juror	Invia un'email di invito a un giurato.	JWT + Ownership del contest; Resend API .
participant-submit-work	Gestisce l'upload del file .zip del Work e delle immagini di anteprima.	JWT + Appartenenza al contest.
user-delete-account	Elimina un utente e tutti i suoi dati e file associati.	JWT (self-service).
admin-delete-account	Versione amministrativa dell'eliminazione account.	X-API-Key (per Retool).
google-places-search-suggestions	Funge da proxy sicuro per l'API di Google Places.	JWT; Google Places API .
get-latest-apk	Proxy sicuro per il download dell'APK da un repository GitHub privato.	JWT; GitHub API .

Tabella 4.2: Elenco delle principali Edge Functions del sistema.

```

1  // Import e inizializzazione del client di Resend.

```

```

2 // La chiave API viene letta in modo sicuro dai segreti di Supabase.
3 import { Resend } from 'npm:resend';
4 const resend = new Resend(Deno.env.get('RESEND_API_KEY'));
5
6 // ... (all'interno della funzione Deno.serve, dopo aver validato l'input
7 // e aver creato il record di invito nel database)
8
9 try {
10   // ...
11   // Passaggio 5: Inserisce il nuovo invito nel DB e ottiene il token
   generato.
12   const { data: newInvitation, error: insertError } = await supabaseAdmin
13     .from('juror_invitations')
14     .insert({ contest_id, jury_id, email })
15     .select()
16     .single();
17
18   if (insertError) throw insertError;
19
20   // Passaggio 6: Recupera i nomi del contest e della giuria per
   personalizzare l'email.
21   const { data: contest } = await supabaseAdmin.from('contests').select('
   name').eq('id', contest_id).single();
22   const { data: jury } = await supabaseAdmin.from('juries').select('name'
   ).eq('id', jury_id).single();
23
24   // Passaggio 7: Invia l'email di invito utilizzando il client di Resend
   .
25   // L'operazione è asincrona.
26   await resend.emails.send({
27     from: "Swift Contest <noreply@swiftcontest.com>",
28     to: [email],
29     subject: `Invitation to join the jury for "${contest.name}"`,
30     // Il corpo dell'email è un template HTML che include i dati dinamici
   .
31     html: `
32       <div>
33         <h1>Invitation to Swift Contest</h1>
34         <p>You have received an invitation to join the "<strong>${jury.name
   }</strong>" jury for the contest "<strong>${contest.name}</strong>".</p>
35         <p>Use this token in the app to accept:<br>
36         <strong>${newInvitation.token}</strong>
37       </p>
38     </div>
39     `,
40   });
41

```

```

42 // Passaggio 8: Se l'invio ha successo, restituisce l'invito creato al
    client.
43 return new Response(JSON.stringify(newInvitation), {
44     headers: { ...corsHeaders, "Content-Type": "application/json" },
45     status: 201, // Created
46 });
47
48 } catch (error) {
49     // Se l'invio dell'email o qualsiasi operazione precedente fallisce,
50     // l'errore viene catturato e registrato, e viene restituita una
    risposta di errore.
51     console.error("Error sending invitation:", error.message);
52     return new Response(JSON.stringify({ error: error.message }), {
53         headers: { ...corsHeaders, "Content-Type": "application/json" },
54         status: 500,
55     });
56 }

```

Listing 4.8: Estratto dalla Edge Function organizer-invite-juror che mostra l'invocazione dell'API di Resend per l'invio di email.

4.5 Architettura di Sicurezza: Principio del Minimo Privilegio e Accesso Mediato

La sicurezza è un requisito trasversale che ha guidato l'intera progettazione del backend. È stato adottato un rigoroso modello basato sul **Principio del Minimo Privilegio (PoLP)**, il cui assunto fondamentale è negare l'accesso per default e concederlo solo in modo esplicito e controllato.

4.5.1 Il Principio del Minimo Privilegio e l'API come Gatekeeper

Tutte le tabelle del database hanno la **Row Level Security (RLS)** abilitata, con una policy di DENY implicita per tutte le operazioni. Questo significa che la chiave API usata dal client Flutter (anon_key) non ha, di per sé, alcun permesso di leggere o scrivere dati.

L'accesso ai dati è quindi interamente mediato da un'API sicura, composta da:

- **Funzioni RPC SECURITY DEFINER:** per la maggior parte delle operazioni sul database.
- **Edge Functions:** per operazioni complesse e interazioni con servizi esterni.

Questa architettura garantisce che tutta la logica di autorizzazione (es. "l'utente è l'organizzatore di questo contest?") sia centralizzata e verificata lato server, aderendo al principio del minimo privilegio.

4.5.2 L'Eccezione Necessaria: le Policy RLS per il Servizio Realtime

Esiste un'eccezione mirata a questa regola. Il servizio **Supabase Realtime**, che permette al client di sottoscrivere modifiche al database in tempo reale, basa il suo meccanismo di autorizzazione proprio sulle policy RLS della tabella. Per permettere a un client di "ascoltare" le modifiche su una tabella, è necessario definire una policy di SELECT che gli conceda il permesso di lettura su quelle specifiche righe.

Questo ha richiesto l'implementazione di policy RLS di tipo SELECT esclusivamente per le due tabelle esposte al servizio Realtime: `messages` e `voting_sessions`. Questo è stato un adattamento necessario per integrare una funzionalità chiave, mantenendo la massima sicurezza su tutte le altre operazioni e tabelle.

Il seguente listato mostra la policy implementata per la tabella `voting_sessions`, un esempio significativo di come l'autorizzazione sia stata definita in modo granulare anche in questo caso eccezionale.

```
1  -- Abilita l'accesso in lettura alla tabella `voting_sessions`
2  -- solo per gli utenti autorizzati a ricevere aggiornamenti in tempo
   reale.
3  CREATE POLICY "Allow read access to organizers and involved jurors"
4  ON public.voting_sessions
5  FOR SELECT
6  USING (
7  -- Condizione 1: L'utente è l'organizzatore del contest a cui la sessione
   appartiene.
8  (EXISTS (
9  SELECT 1 FROM public.contests c
10 WHERE c.id = voting_sessions.contest_id AND c.organizer_id = auth.uid()
11 ))
12 OR
13 -- Condizione 2: L'utente è un giurato che partecipa a questa specifica
   sessione di voto.
14 (EXISTS (
15 SELECT 1 FROM public.voting_session_jurors vsj
16 WHERE vsj.voting_session_id = voting_sessions.id AND vsj.juror_id = auth.
   uid()
17 ))
18 );
```

Listing 4.9: Policy RLS di tipo SELECT per la tabella `voting_sessions`, necessaria per il servizio Realtime.

4.5.3 Misure di Sicurezza Aggiuntive

Oltre al modello ibrido RLS/RPC, la sicurezza del sistema è rafforzata da altre misure:

- **Protezione delle API Esterne:** le chiavi segrete per servizi come Resend e Google Places sono memorizzate in Supabase e accessibili solo dalle Edge Functions.
- **Protezione dei file:** nessun file nello Storage è pubblico. L'accesso avviene solo tramite *signed URL* temporanei generati lato server.

Gestione dei Giurati Semplici: il Pattern dell'Autenticazione Anonima

Una delle sfide più complesse è stata quella di permettere a un *Simple Juror* di votare in modo sicuro senza obbligarlo a creare un account permanente. Risolvere questo problema solo con un token temporaneo sarebbe stato insicuro, poiché non avrebbe permesso di tracciare in modo affidabile se l'utente avesse già votato.

Per risolvere questa sfida, è stato implementato un pattern basato sull'**autenticazione anonima** offerta da Supabase. Questo approccio è un'applicazione diretta del Principio del Minimo Privilegio. Il flusso è il seguente:

1. L'utente inserisce il token della giuria semplice. La RPC `juror_access_voting_as_simple_juror` valida il token.
2. Se il token è valido, l'applicazione chiede all'utente di inserire il proprio nome e, subito dopo, il client esegue una chiamata a Supabase per effettuare un **sign-in anonimo**.
3. Supabase Auth crea un account utente a tutti gli effetti, ma marcato come "anonimo", e restituisce un JWT valido. Da questo momento, anche l'utente anonimo ha un `auth.uid()` univoco.
4. L'AuthBloc nel client entra in uno stato "autenticato ma anonimo".
5. L'AuthGuard, riconoscendo questo stato, applica una logica di navigazione restrittiva: permette l'accesso solo alle pagine strettamente necessarie per la votazione (es. la procedura di voto), bloccando tutte le altre sezioni dell'app (es. impostazioni, dashboard).
6. Al termine della sessione, quando l'utente effettua il logout, viene invocata la Edge Function `user-delete-account`. Poiché l'utente possiede un JWT valido, è autorizzato a chiamare questa funzione, che elimina in modo sicuro e permanente l'account anonimo appena utilizzato.

Questo pattern offre enormi vantaggi:

- **Esperienza Utente Fluida:** L'utente non percepisce la creazione di un account, ma solo l'inserimento del proprio nome.
- **Sicurezza Robusta:** Ogni azione del giurato semplice (come l'invio dei voti) è autenticata e legata a un `uid` univoco, prevenendo abusi.

- **Riutilizzo dell'Architettura:** Permette di riutilizzare la stessa logica di sicurezza (RLS, RPC che controllano `auth.uid()`, `AuthGuard`) sia per gli utenti normali che per quelli anonimi, senza dover creare un sistema di autorizzazione parallelo e meno sicuro.

4.6 Progettazione e Implementazione degli Strumenti di Amministrazione

Oltre all'applicazione destinata agli utenti finali, un sistema software robusto richiede strumenti adeguati per la sua gestione, moderazione e manutenzione. Per Swift Contest, si è deciso di non sviluppare un'applicazione admin da zero, ma di utilizzare **Retool**, una piattaforma low-code specializzata nella creazione rapida di strumenti interni.

4.6.1 Esigenza di un Pannello di Amministrazione

La necessità di un pannello di amministrazione nasce da requisiti operativi critici non gestibili dagli utenti standard:

- **Gestione Utenti:** la capacità di visualizzare tutti gli utenti registrati, verificarne l'attività e, in caso di abuso, procedere con l'eliminazione dell'account.
- **Moderazione Contenuti:** la possibilità per un amministratore di ispezionare e, se necessario, rimuovere forzatamente un contest che viola i termini di servizio.

4.6.2 Scelta di Retool per lo Sviluppo Rapido

La scelta di Retool è stata strategica e motivata da:

- **Velocità di Sviluppo:** ha permesso di costruire un'interfaccia admin funzionale in poche ore, invece che in settimane, concentrando le risorse di sviluppo sull'applicazione principale.
- **Integrazione Nativa:** si integra nativamente con database PostgreSQL e API REST, permettendo di interfacciarsi facilmente con l'infrastruttura Supabase.
- **Sicurezza Configurabile:** offre meccanismi per gestire segreti (come le chiavi API) e configurare le chiamate in modo sicuro.

4.6.3 Architettura di Comunicazione Sicura tra Retool e Supabase

La sicurezza è stata la priorità nella progettazione dell'integrazione. L'accesso ai dati non è mai diretto, ma mediato da un'API sicura esposta dal backend Supabase, con due meccanismi distinti:

Accesso in Lettura tramite Utente Dedicato e Funzioni RPC

Per le operazioni di sola lettura (es. visualizzare la lista di utenti), è stato creato un utente PostgreSQL dedicato, `retool`, con permessi minimi. Le query da Retool invocano funzioni RPC con prefisso `admin_*`, che al loro interno verificano che il chiamante (`session_user`) sia esattamente `retool`, bloccando qualsiasi altro tentativo di accesso.

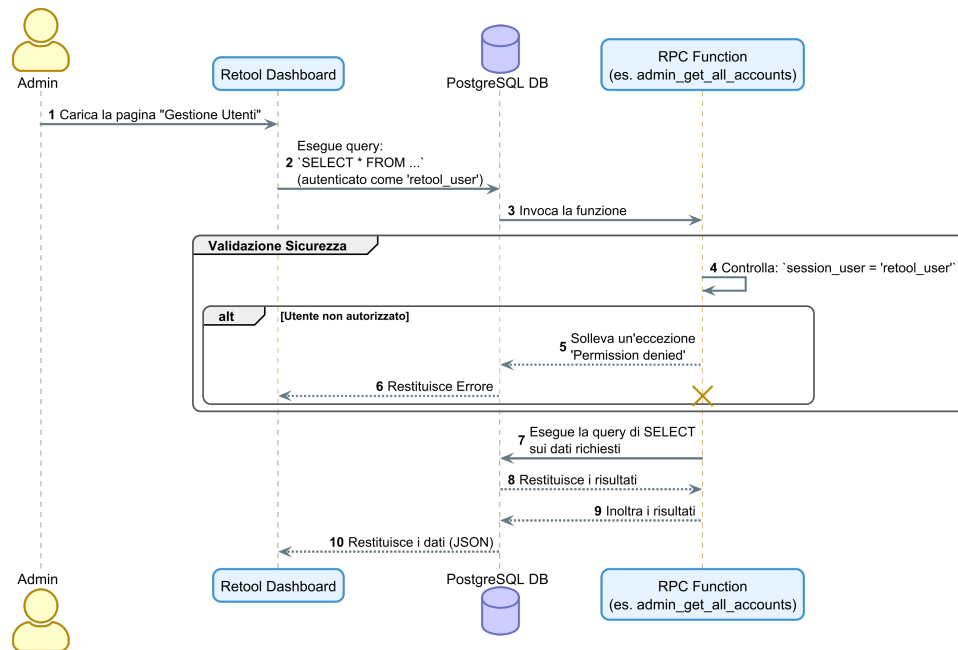


Figura 4.4:

```
1  -- Estratto da una funzione come 'admin_get_all_accounts()'
2
3  CREATE OR REPLACE FUNCTION admin_get_all_accounts()
4  RETURNS SETOF profiles -- o un altro tipo di ritorno
5  LANGUAGE plpgsql
6  SECURITY DEFINER
7  SET search_path = public;
8  AS $$
9  BEGIN
10     -- --- 1. CONTROLLO DI SICUREZZA ---
11     -- Questa è la prima operazione eseguita dalla funzione.
12     -- 'session_user' è una variabile speciale di PostgreSQL che contiene
13     -- il nome dell'utente della connessione corrente (in questo caso, '
14     retool_user').
15
16     IF session_user <> 'retool' THEN
17         -- Se il chiamante non è l'utente 'retool' designato,
18         -- la funzione solleva un'eccezione e interrompe immediatamente l'
19         esecuzione.
```

```

18 RAISE EXCEPTION 'Permission denied: This function can only be called by
    the admin user.';
19 END IF;
20
21 -- --- 2. ESECUZIONE DELLA LOGICA ---
22 -- Se il controllo di sicurezza passa, la funzione procede
23 -- con l'esecuzione della query per recuperare i dati.
24 RETURN QUERY
25 SELECT * FROM public.profiles;
26
27 END;
28 $$;

```

Listing 4.10: Validazione del chiamante all'interno di una funzione RPC per l'amministrazione.

Operazioni di Scrittura tramite Edge Functions e API Key

Per le operazioni che richiedono un'orchestrazione complessa o l'interazione con servizi esterni al database, come l'invio di email o la manipolazione di file nello Storage, sono state implementate delle **Edge Functions**. Funzioni critiche come `admin-delete-account` e `admin-delete-contest` rientrano in questa categoria, poiché devono non solo modificare i record del database, ma anche eliminare tutti i file associati (immagini, file) dallo Storage per garantire una pulizia completa.

Queste funzioni sono protette da un meccanismo di autenticazione server-to-server per garantire che possano essere invocate solo da un client autorizzato come Retool:

1. Retool, nell'eseguire l'azione, invia una richiesta HTTP POST alla Edge Function, includendo un header personalizzato: `X-API-Key`.
2. Il valore di questa chiave è un segreto memorizzato sia in Retool che nel in Supabase.
3. La Edge Function, come primo passo, confronta la chiave ricevuta con quella memorizzata nei suoi segreti. Se non corrispondono, la richiesta viene immediatamente respinta con un errore 401 (Unauthorized).

Questo approccio garantisce che solo un'applicazione autorizzata possa invocare queste potenti funzioni, proteggendo il sistema da accessi impropri.

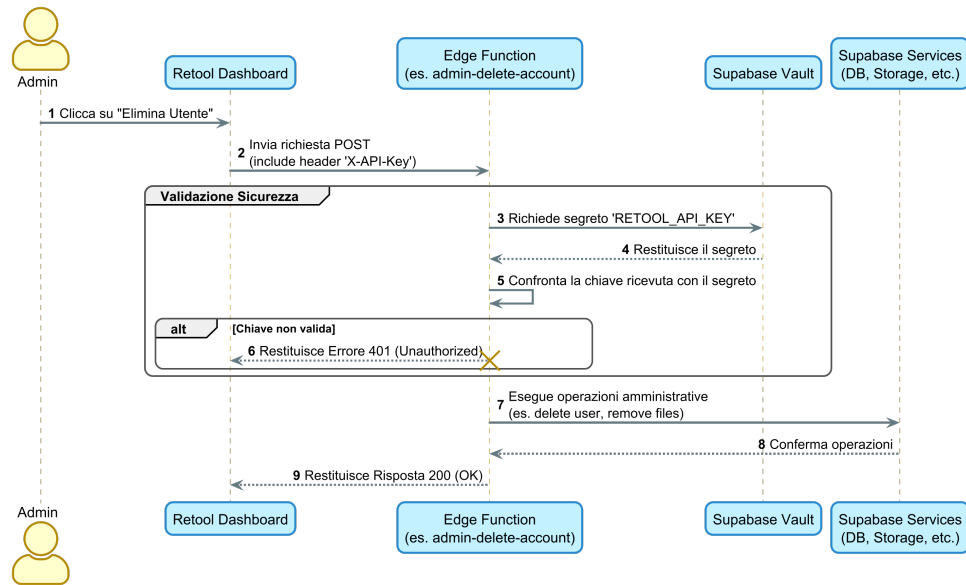


Figura 4.5:

```

1 // Estratto da una Edge Function come 'admin-delete-account/index.ts'
2
3 // ... (all'interno della funzione Deno.serve)
4 try {
5   // --- 1. API KEY AUTHORIZATION ---
6   // Questo pattern è usato per tutte le chiamate server-to-server da
7   // Retool.
8
9   // Legge la chiave API inviata dal client (Retool) nell'header.
10  const apiKey = req.headers.get('X-API-Key');
11
12  // Recupera la chiave segreta memorizzata in modo sicuro nel Vault di
13  // Supabase.
14  const secretKey = Deno.env.get('RETOOL_API_KEY');
15
16  // Primo controllo: verifica che il segreto sia stato configurato.
17  if (!secretKey) {
18    throw new Error("RETOOL_API_KEY is not set in Supabase secrets.");
19  }
20
21  // Secondo controllo: confronta la chiave ricevuta con il segreto.
22  // Se non corrispondono, la richiesta viene immediatamente respinta.
23  if (apiKey !== secretKey) {
24    return new Response(
25      JSON.stringify({ error: "Unauthorized. Invalid API Key." }),
26      {
27        headers: { ...corsHeaders, "Content-Type": "application/json" },
28        status: 401, // 401 Unauthorized
29      }
30    );
31  }
32 }
  
```

```

28     );
29 }
30
31 // --- 2. PROSECUZIONE DELLA LOGICA ---
32 // Se la validazione ha successo, la funzione procede con
33 // l'esecuzione delle operazioni amministrative.
34
35 // ... (logica per eliminare l'utente, i file, ecc.)
36
37 } catch (error) {
38     // ... (gestione degli errori)
39 }

```

Listing 4.11: Validazione della chiave API all'interno di una Edge Function amministrativa.

4.6.4 Funzionalità Implementate nel Pannello

La dashboard sviluppata in Retool fornisce all'amministratore le seguenti funzionalità chiave:

- Una tabella per la visualizzazione e la ricerca di tutti gli utenti registrati.
- Una vista di dettaglio per un singolo utente, che mostra i contest a cui è associato e i suoi ruoli.
- Un pulsante "Elimina Utente", che invoca in modo sicuro la Edge Function `admin-delete-account`.
- Una tabella di tutti i contest, con un pulsante "Elimina Contest" che invoca la Edge Function `admin-delete-contest`.

Capitolo 5

Distribuzione e Prodotto finale

5.1 Strategia di distribuzione

La fase di distribuzione ha avuto come obiettivo non solo il rilascio tecnico dell'applicazione, ma anche la validazione della sua utilizzabilità in un ambiente che simula condizioni reali di produzione. Per **Swift Contest** è stato scelto un approccio multiplatforma che copre i principali punti di accesso per i diversi tipi di utenti:

- **Applicazione Web:** distribuita su **Vercel**, per garantire accessibilità universale da qualsiasi browser moderno, sia desktop che mobile.
- **Applicazione Android:** distribuita come pacchetto APK, reso disponibile tramite un meccanismo di download sicuro per un'esperienza d'uso ottimizzata su dispositivi mobili.
- **Infrastruttura Backend:** interamente gestita su **Supabase Cloud**, che ospita il database, lo storage, le funzioni serverless e i servizi realtime.
- **Pannello di Amministrazione:** sviluppato come strumento interno su **Retool** e interfacciato in modo sicuro con il backend, per le attività di moderazione e manutenzione.

Questa strategia ibrida consente di bilanciare la facilità di accesso (garantita dalla versione Web) con un'esperienza d'uso ottimizzata (offerta dalla versione Android), mantenendo al contempo un backend centralizzato, sicuro e scalabile.

5.3 Distribuzione Android

Per la piattaforma Android, è stata prodotta una **release in formato APK**. Data la natura prototipale del progetto, si è evitato il complesso processo di pubblicazione sul Google Play Store, optando per un approccio di distribuzione controllata che garantisce comunque sicurezza e facilità di aggiornamento:

1. L'APK viene caricato come *release asset* su un repository GitHub privato.
2. Un'apposita Edge Function, `get-latest-apk`, è stata creata su Supabase per fungere da proxy sicuro. Questa funzione utilizza un Personal Access Token (PAT) di GitHub, memorizzato in modo sicuro in Supabase, per autenticarsi all'API di GitHub e scaricare l'asset.
3. L'applicazione web offre un pulsante "Download for Android" che invoca questa Edge Function. L'utente riceve così l'ultima versione dell'APK direttamente dal backend, senza che il repository GitHub o le credenziali di accesso vengano mai esposte.

Questa soluzione rappresenta un eccellente compromesso tra sicurezza, controllo delle versioni e facilità di distribuzione per un contesto non commerciale.

5.4 Backend e Supabase

L'intera infrastruttura backend è gestita su Supabase. Il processo di deployment del database è basato su migrazioni SQL, seguendo le pratiche di **Infrastructure as Code (IaC)**. Ogni modifica allo schema del database, alle policy RLS o alle funzioni RPC è definita in un file `.sql` all'interno della cartella `supabase/migrations`. La CLI di Supabase viene utilizzata per applicare queste migrazioni in modo sequenziale e versionato, garantendo che l'ambiente di produzione sia sempre allineato con la definizione del codice. Le Edge Functions, scritte in TypeScript, vengono anch'esse deployate tramite la CLI.

5.5 Dashboard amministrativa (Retool)

Per le attività di amministrazione avanzata, come la moderazione degli utenti e la gestione dei contenuti, è stata creata una dashboard interna utilizzando **Retool**. Questa interfaccia, non esposta agli utenti finali, comunica con il backend Supabase attraverso l'API sicura descritta nel capitolo precedente, utilizzando un utente di database dedicato per le letture e Edge Functions protette da API-Key per le operazioni di scrittura.

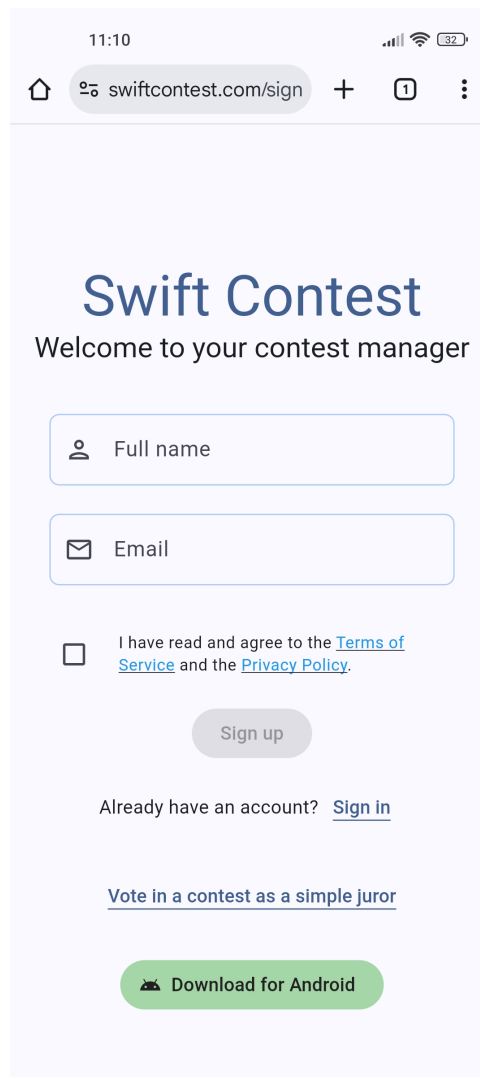


Figura 5.2: Pagina di accesso dell'applicazione web distribuita su Vercel con pulsante di download.

5.6 Prodotto finale: Flussi Operativi e Interfaccia Utente

Il prodotto finale offre interfacce e flussi operativi specifici e ottimizzati per ogni attore del sistema, garantendo un'esperienza utente coerente tra la versione web e quella Android. Le sezioni seguenti descrivono il ciclo di vita completo di un contest attraverso i percorsi dei suoi protagonisti.

5.6.1 Flusso dell'Organizzatore

L'organizzatore è l'utente con il set di funzionalità più ricco, responsabile dell'intero ciclo di vita del contest.

Creazione e Configurazione Dopo l'autenticazione, l'organizzatore accede a una dashboard personale dove può visualizzare e gestire i contest da lui creati. Il flusso principale lo guida

attraverso la creazione di un nuovo contest tramite un form multi-step in cui inserisce dati principali (nome, descrizione, data), periodo di sottomissione, luogo e immagini rappresentative. Una volta creato il contest, accede a un pannello di gestione dettagliato dove può:

- **Creare le giurie**, specificando se sono di tipo *appointed* (per giurati nominati) o *simple* (per giurati semplici con token).
- **Invitare i partecipanti e i giurati nominati**, inserendo i loro indirizzi email. Apposite Edge Functions gestiscono l'invio sicuro delle email di invito.
- **Configurare il form di voto** associato a ciascuna giuria, definendo i campi testuali e slider con i relativi scope (header, participant, footer).

Gestione della Votazione e dei Risultati Terminato il periodo di sottomissione, l'organizzatore avvia una *VotingSession*, creando lo snapshot immutabile dei dati. Una volta che tutti i giurati hanno votato, chiude la sessione. Ora ha a disposizione i risultati per ciascuna giuria, che può esportare in formato Excel (xlsx). Ha inoltre la possibilità di generare una classifica per ciascuna giuria selezionando i campi required, di tipo slider, e con scope participant, che desidera considerare. La classifica viene visualizzata in app e può essere scaricata in formato pdf. A questo punto l'organizzatore può raffinare le classifiche esternamente, ad esempio può generare un'unica classifica considerando quelle di tutte le giurie, e pubblicarla in formato pdf, rendendola disponibile per il download per i partecipanti e giurati nominati facenti parte del contest.

5.6.2 Flusso del Partecipante

Il partecipante, una volta accettato l'invito a un concorso, ha un percorso chiaro e focalizzato. La sua azione principale è la **sottomissione del proprio Work**. Un form dedicato lo guida nel caricamento di un pacchetto .zip contenente il materiale principale e delle immagini di anteprima, che verranno usate durante la votazione. Al termine del concorso, può accedere alla sezione dei risultati per visualizzare la classifica finale, se pubblicata dall'organizzatore.

5.6.3 Flusso del Giurato (Nominato e Semplice)

Il sistema supporta due modalità di accesso per i giurati, garantendo flessibilità.

- Il **giurato nominato (*appointed*)** accede alla sua dashboard personale e visualizza l'elenco dei contest a cui è stato assegnato.
- Il **giurato semplice (*simple juror*)**, invece, accede direttamente alla procedura di voto tramite un token fornito dall'organizzatore, senza bisogno di un account.

Per entrambi, l'interfaccia di voto è identica e li guida attraverso una procedura strutturata: per ogni *Work* in gara, viene presentato il materiale di anteprima e il *VotingForm* da compilare. Il sistema garantisce che i voti possano essere inviati una sola volta per sessione.

5.6.4 Flusso delle Notifiche

Per mantenere tutti gli utenti informati, il sistema include una **inbox** interna, accessibile dalla barra di navigazione principale. Qui, tutti gli utenti autenticati ricevono notifiche generate automaticamente per eventi rilevanti, come la ricezione di un invito, la cancellazione di un contest o la pubblicazione di una classifica. La ricezione di nuovi messaggi avviene in tempo reale, segnalata da un indicatore visivo sull'icona della inbox, grazie all'integrazione con il servizio Realtime di Supabase.

Capitolo 6

Conclusioni

6.1 Sintesi del lavoro svolto e risultati ottenuti

Il progetto di tesi ha portato alla progettazione, implementazione e distribuzione di **Swift Contest**, una piattaforma software completa e funzionante per la gestione di concorsi, che raggiunge con successo tutti gli obiettivi prefissati in fase di analisi. È stato dimostrato come, attraverso l'integrazione di un ecosistema tecnologico moderno e accessibile, sia possibile costruire un sistema complesso, sicuro e scalabile, capace di risolvere le inefficienze e le criticità dei processi di gestione tradizionali.

I principali risultati ottenuti possono essere riassunti come segue:

- **Realizzazione di un'applicazione cross-platform unificata:** utilizzando Flutter, è stato sviluppato un unico client che offre un'esperienza utente coerente e funzionale sia su piattaforme web che mobile (Android), validando l'efficacia di un approccio a codebase singola per ridurre i tempi di sviluppo e la complessità di manutenzione.
- **Implementazione di un'architettura backend sicura e robusta:** è stato costruito un backend basato sul Principio del Minimo Privilegio, dove l'accesso ai dati è interamente mediato da un'API controllata, proteggendo l'integrità del sistema.
- **Creazione di un sistema di votazione affidabile e trasparente:** il meccanismo di *snapshot* delle sessioni di voto garantisce l'immutabilità dei dati e l'auditabilità del processo, risolvendo una delle principali criticità dei sistemi di valutazione manuali.
- **Validazione di un'infrastruttura efficiente e sostenibile:** l'architettura basata su servizi cloud (Supabase, Vercel) e l'uso mirato di risorse onerose come il *realtime* dimostrano la fattibilità di costruire soluzioni complesse mantenendo i costi operativi contenuti, in linea con i piani gratuiti o a basso costo offerti dai provider.

Il prototipo realizzato non è solo una dimostrazione tecnica, ma si presenta come un prodotto maturo e coeso, già utilizzabile in contesti reali a piccola e media scala.

6.2 Limiti e analisi critica

Un'analisi critica e onesta del lavoro svolto è fondamentale per comprenderne appieno il valore e i margini di miglioramento. Nonostante i risultati positivi, il progetto presenta alcuni limiti, intrinseci a un elaborato sviluppato in un contesto accademico con vincoli temporali definiti.

- **Logica di ranking semplificata:** l'attuale meccanismo di calcolo delle classifiche, basato sulla semplice somma dei punteggi dei campi *slider*, è funzionale ma basilare. Manca la possibilità di introdurre regole più complesse, come l'assegnazione di pesi diversi ai campi di voto o l'applicazione di algoritmi di scarto per i voti estremi (es. eliminare il voto più alto e più basso), limitandone l'applicabilità in contesti con regolamenti di valutazione più sofisticati.
- **Assenza di versioning dei form di voto:** la stabilità delle votazioni è garantita tramite snapshot, ma non esiste un vero sistema di versionamento per i *VotingForm*. Una volta che un form è stato usato, modificarlo potrebbe creare incongruenze se lo si volesse riutilizzare in futuro. Una gestione delle versioni permetterebbe agli organizzatori di creare e archiviare template di form riutilizzabili.
- **Osservabilità e monitoraggio limitati:** il sistema non integra strumenti avanzati di logging, metriche o tracing distribuito. In un ambiente di produzione su larga scala, l'assenza di questi strumenti renderebbe più complessa l'analisi delle performance, il debug di errori specifici e il monitoraggio proattivo della salute del sistema.
- **Test automatici non esaustivi:** sebbene la logica di business nei BLoC sia stata coperta da test di unità, il progetto beneficerebbe enormemente dall'aggiunta di una suite di test più completa, includendo test di integrazione per verificare l'interazione tra client e backend e test end-to-end per simulare i flussi utente completi.

6.3 Prospettive e sviluppi futuri

Il progetto attuale costituisce una solida base per numerosi e interessanti sviluppi futuri, che potrebbero trasformare Swift Contest da un prototipo avanzato a un prodotto commerciale completo e competitivo.

- **Ranking avanzati e configurabili:** lo sviluppo più immediato e di maggior valore sarebbe l'introduzione di un sistema che permetta agli organizzatori di definire regole di calcolo personalizzate per le classifiche, aumentando la flessibilità della piattaforma.

- **Supporto per sottomissioni multiple (Multi-Work):** estendere il modello dati per permettere ai partecipanti di inviare più di un *Work* all'interno dello stesso concorso è una funzionalità molto richiesta, specialmente in ambiti come la fotografia o la poesia.
- **Integrazione di notifiche push:** affiancare alle notifiche inbox un sistema di notifiche push (es. tramite Firebase Cloud Messaging o un altro provider) aumenterebbe significativamente il coinvolgimento e la reattività degli utenti, notificandoli istantaneamente di eventi importanti.
- **Hardening della sicurezza:** sebbene l'architettura sia sicura, potrebbe essere ulteriormente rafforzata con meccanismi di *rate limiting* sulle Edge Functions per prevenire abusi e attacchi di tipo DoS, e con policy RLS ancora più granulari.
- **Internazionalizzazione (i18n) e accessibilità (a11y):** la traduzione dell'interfaccia utente in più lingue e il miglioramento del supporto per le tecnologie assistive (es. screen reader) sono passi fondamentali per rendere l'applicazione accessibile a un pubblico globale e inclusivo.
- **Distribuzione ufficiale su App Store:** per completare la visione cross-platform, un passo futuro sarebbe quello di creare una versione per iOS e pubblicare entrambe le applicazioni mobile sugli store ufficiali (Google Play Store e Apple App Store), semplificandone la distribuzione e gli aggiornamenti.

6.4 Considerazioni finali

Swift Contest rappresenta un esempio concreto di come un'architettura software ben ponderata, unita a un ecosistema tecnologico moderno e accessibile, possa portare alla creazione di un sistema complesso, sicuro e di valore. La scelta di combinare la flessibilità di Flutter con la potenza e la sicurezza di Supabase si è rivelata vincente, permettendo di raggiungere un ottimo equilibrio tra rapidità di sviluppo, ricchezza funzionale e sostenibilità dei costi.

Il progetto non si esaurisce con la presente tesi; esso costituisce una base di partenza robusta e scalabile per future iterazioni e, potenzialmente, per applicazioni in contesti reali. Il lavoro svolto conferma la validità dell'approccio architetturale adottato e apre prospettive interessanti nel campo delle piattaforme collaborative e dei sistemi di votazione digitale, dimostrando che è possibile realizzare prototipi di alta qualità anche con le risorse a disposizione di uno studente o di un piccolo team di sviluppo.